The Sustainability Face of Automated Program Repair Tools

MATIAS MARTINEZ, Universitat Politècnica de Catalunya, Spain
SILVERIO MARTÍNEZ-FERNÁNDEZ, Universitat Politècnica de Catalunya, Spain
XAVIER FRANCH, Universitat Politècnica de Catalunya, Spain

Automated program repair (APR) aims to automatize the process of repairing software bugs in order to reduce the cost of maintaining software programs. While APR accuracy has significantly improved in recent years, its energy impact remains unstudied. The field of green software research aims to measure the energy consumption required to develop, maintain, and use software products. Our main goal is to define the foundation for measuring the energy consumption of the APR activity. We state that an environmentally *sustainable (or green) APR tool* achieves a good balance between the ability to correctly repair bugs and the amount of energy consumed during such process. We measure the energy consumption of ten traditional APR tools for Java and eleven fine-tuned Large-Language Models (LLM) trying to repair real bugs from Defects4J. The results of this study show the existing trade-off between energy consumption and repairability. In particular, APR tools such as TBar and RepairLlama repair more bugs than other approaches at the expense of a higher energy consumption. Other tools, such as SimFix and the LLM CodeT5-Large, provide a good trade-off between energy consumption and repairability. We also present guidelines consisting of a set of recommendations for developing greener APR.

CCS Concepts: • **Hardware** → **Power and energy**; • **Software and its engineering** → **Software maintenance tools**.

Additional Key Words and Phrases: Automated Program Repair, Software Sustainability, Energy Consumption of Software Tools, Green Computing

1 INTRODUCTION

In the last decade, following current societal needs, software sustainability has emerged as research field [1–3]. It consists of different dimensions, including environmental sustainability, human sustainability, and economic sustainability. In this paper, we focus on environmental sustainability, defined as *"how software product development, maintenance, and use affect energy consumption and the consumption of other natural resources. [...] This dimension is also known as Green Software"* [4].

The study of environmental sustainability is of paramount importance in the realm of automated software engineering, i.e. the application of computation to software engineering activities with the objective of partially or fully automating these activities, thereby significantly increasing both quality and productivity [5]. The reason is that such computations demand high computational resources, especially nowadays: the advent of Big Data, Internet of Things, Machine Learning, Generative AI and other contemporary methods and technologies, supported by high-performance cloud-based infrastructure, allows researchers to design and implement complex, sophisticated automated solutions that were not feasible in the past, which ultimately implies high levels of energy consumption.

Authors' Contact Information: Matias Martinez, Universitat Politècnica de Catalunya, Barcelona, Spain, matias.martinez@upc.edu; Silverio Martínez-Fernández, Universitat Politècnica de Catalunya, Barcelona, Spain, silverio.martinez@upc.edu; Xavier Franch, Universitat Politècnica de Catalunya, Barcelona, Spain, xavier.franch@upc.edu.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
© 2025 Copyright held by the owner/author(s).
ACM 1557-7392/2025/6-ART
https://doi.org/10.1145/3744900

Just Accepted

One of these automation-prone activities is bug fixing through automated program repair (APR) tools. APR is becoming popular due to the high economic cost that fixing bugs entails [6, 7]. APR tools take as input a buggy program and deliver one or more patches that repair the bugs, when they are found. In recent years, numerous APR tools have been capable of automatically repairing real bugs (e.g., [8–10]), mainly from open-source libraries [11–13]. Furthermore, APR has started to be adopted in industry [14–16].

To evaluate APR tools, researchers usually report two metrics [17]: *a)* Total number of bugs that are repaired with a *plausible* patch (that is, a patch valid w.r.t. the correctness oracle), and *b)* Total number of bugs repaired with at least one *correct* patch.

However, to our knowledge, **there is no previous work in the APR area that focuses on the study of the energy consumption of APR tools**. The APR research community has been oriented towards accuracy, with a strong focus on obtaining state-of-the-art results, as happened in the AI community [18]. According to current societal demands and the computational demanding nature of state-of-the-art techniques, we argue that **APR researchers should also consider energy consumption as a main driver when designing and implementing APR tools**, carefully analyzing whether a minor gain in accuracy justifies high levels of energy consumption. The absence of methodologies for measuring energy consumption and benchmarkings in APR makes it challenging to evaluate how innovations in APR impact the overall energy consumption of the repair process. This paper aims at addressing this gap.

The inclusion of energy as a driver of APR research is important for several reasons. First, APR tools use expensive correctness oracles, executed each time a candidate patch is synthesized. Test suites are examples of them. As APR tools synthesize a considerable amount of patches before finding the correct one [19], the amount of energy up to that point could be considered wasteful. Second, even with the impressive progress of the field, state-of-the-art APRs do not achieve high accuracy and are complementary (as shown, for example, in Fig. 2 from [20]). This means that, in order to achieve high accuracy, a practitioner aiming to deploy APR would need to consider not just one tool, but several, increasing the energy required to repair a bug. Third, APR tools could be triggered regularly; for example, a setup that connects APRs with CI/CD (e.g., via a bot [21]) would require to invoke APRs tools for each failed build caused by test cases with failure or error. For example, the study by Moritz et al. [22] on Travis CI builds reports a median of 10.3% for builds with failed tests in Java projects. Triggering APRs for each of these failed builds would require a large amount of repair attempt execution. Unfortunately, the energy consumed by these repair attempts is unknown. To achieve green software development, we believe that it is essential to understand the energy consumption of APR activity. This work is a first step in that direction.

The ultimate goal of our research, expressed following GQM [23] is to: Analyze *APR tools* with the purpose of *measurement and analysis* with respect to *energy consumption* from the point of view of *software developers* in the context of *bug repair*.

To achieve this goal, we first measure the energy consumed by APR tools to find plausible and correct patches. We focus on two families of APRs: *a) Traditional* tools (i.e., search-, constraint- and template-based), and *b) Large-Language Model* (LLM)-based tools. We also focus on the correctness of generated patches, in order to quantify the energy 'wasted' on generating *overfitting* (incorrect) patches.

Then, we study the relationship between energy consumed and two metrics studied in previous work as proxies of *repair efficiency*, which is one aspect of the APR performance [19]: *a)* Repair time, studied by [24] in the context of APR, and *b)* Number of candidate patches (NCP), defined by [25], and studied by Liu et al. [19] in the context of APR. With this study, we want to know whether these two metrics can be used as a proxy for energy consumption. In addition, we study whether repairing hard and important bugs requires more energy. For this, we use the data provided by Motwani et al. [26].

We evaluate on the Defects4J benchmark [11] the energy consumption of ten publicly available traditional repair tools (including TBar [27], Prapr [24] and SimFix [28]), ten fine-tuned large language models (including InCoder [29] and CodeGen [30]) used by Jiang et al. [31], and RepairLlama [32], a repair approach based on

Just Accepted

CodeLlama [33], which introduces LoRA [34], a state-of-the-art fine-tuning technique named PEFT (parameter-efficient fine-tuning), for program repair.

The results of this experiment show a moderate positive correlation between energy consumed and number of bugs correctly repaired. The traditional approach TBar, based on a collection of existing templates, is the most energy-consuming approach to repair Defects4J bugs (118 kWh). In total, 28 bugs were correctly fixed by TBar, more than any other traditional APR tool.

Among LLM-based tools, the largest models (those with ≈ 6-7 billion parameters) consume more energy due to their hardware requirements (they require several GPUs). In particular, RepairLlama consumes 1,263 Wh to repair 61 bugs correctly. Tools based on LLM with similar size (6 billons parameters) such as InCoder and CodeGen, consumes much less energy than RepairLLama (both ≈820) but also repair less bugs, 37. Smaller models such as CodeT5Large and PLBARTLarge show a good balance between energy (608 Wh and 567 Wh, respectively) and repairability (31 and 30 repaired bugs, respectively).

We also focus on the impact of overfitting patches on the overall energy consumed by the repair tools. In traditional tools, approximately 6% to 40% of the energy is consumed by repair attempts that yield only overfitting patches. In contrast, for LLM-based tools, the energy spent on repair attempts that produce solely overfitting patches ranges from approximately 5% to 17%.

Based on the insights from our research, we propose a set of guidelines with recommendations for developing greener APR. These recommendations cover various types of APR, including LLM-based approaches, and address multiple stages of the repair process, such as patch generation, model training, and patch validation.

The paper continues as follows. Section 2 presents a background on APR and energy consumption. Section 3 presents the methodology to answer the research questions. Section 4 responds to the RQs. Section 5 presents a discussion on this study. Section 6 presents guidelines with recommendations for greener APR. Section 7 presents the threats to validity. Section 8 discusses the limitations of this work. Section 9 concludes the paper and highlights directions for future research.

This is an extended version of a two-page article that introduces the problem of APR and energy consumption [35].

Data availability statement. All data generated in this experiment are available as an appendix at https://github.com/UPCBarcelonaTech/green_apr.

2 BACKGROUND

2.1 Automated program repair

2.1.1 Approaches and tools. During recent years, several APR approaches and tools have been proposed with the goal of automatically fixing bugs in software applications [36, 37]. As a recent controlled experiment with 40 developers shows, proposing correct patches to developers helps successfully repair defects and speeds up debugging time [38].

A family of APR approaches is *test-suite based program repair* approaches [8], which use test cases as a proxy for the program specification. This family is further subdivided into several types. On the one hand, the *traditional*[1] family includes: (a) *Search-based* APRs, such as GenProg [8], PAR [40], SPR [9] and CapGen [41]. These APRs navigate the search space composed of candidate patches to find the solution, i.e. the correct patch. Some of them use information previously extracted from other programs (e.g., [42, 43]) to speed up the search [44] or even in trained models (e.g., [45]) to filter out incorrect candidates; (b) *Semantic-based* APRs, such as SemFix [46], Nopol [47] and Angelix [10], which use symbolic execution and constraint solving.

On the other hand, there exists the *Neural network-based* APR family, which includes, for instance, SequenceR [48], Coconut [49], Recoder [50], SelfAPR [51], KNOD [52] and RewardRepair [20]. These APRs rely on

[1]We adopt the term *traditional* for these approaches, as done by two recent surveys: [39] and [37].

Just Accepted

machine learning to generate patches (e.g., using sequence-to-sequence learning). More recently, researchers have proposed *LLM-based approaches* that, when pre-trained in source code, show good results in fixing bugs [31, 53]. Also, conversational chatbots, such as ChatGPT, are capable of proposing a fixed version of small buggy programs [54].

2.1.2 Adoption of APR. APR has also gained traction in the industry. Practitioners have conducted experiments and implemented APR tools in real-world industrial contexts. Some examples are Meta's tool, Getafix [15], and Bloomberg's tools, Fixie [14] and B-Assist [55]. For instance, an experiment with Bloomberg's software engineers demonstrated that suggestions provided by B-Assist were accepted 74.56% of the time, either directly or after modification, through the GitHub user interface [55]. A recent survey conducted with 337 software practitioners [56] reveals that the majority are either aware of or actively use APR tools. Among the tools mentioned by respondents, the most frequently cited was Astor [57], which includes approaches such as jGenProg and jMutRepair, both studied in our paper. However, the survey also highlighted several disadvantages of APR tools. The most significant concern, reported by 196 respondents, was that these APR tools may overlook rare or uncommon behavior. Additionally, 177 respondents noted that the maintenance cost of APR tools can be high.

2.1.3 Cost and Efficiency of APR. The cost of APR has been studied previously in a few previous works. For example, Le Goues et al. [58] studied the monetary cost of repairing C bugs using GenProg [8]. Other works have studied the efficiency of program repair. For example, Liu et al. [19] use the Number of Candidate Patches (NCP), metric originally used by to measure the efficiency of repair approaches [25]. Nevertheless, beyond the progress made in the field [37, 39], to our knowledge, there are no previous works focusing on the energy consumption of APR. Moreover, even there are different studies on the energy consumption of LLMs, in both training and inference phases, to our knowledge, there is no previous work on the energy profile of LLMs used on program repair (for example, energy of patch inference using LLMs).

2.1.4 Patch Overfitting. Preliminary experiments with APR, such as Smith et al. [59], Martinez et al. [60], Qi et al. [61] and Le et al. [62] have detected one of the initial limitation of APR: a considerable portion of generated patches are incorrect. These patches are known as *Overfitting* patches. Just *Correct* patches are those that correctly repair a bug. Overfitting patches are not accepted by developers.

APR tools rely on test cases from project under repair to identify bugs and validate fixes. Unfortunately, a candidate patch passing all test cases is *Plausible*, but that does not guarantee correctness: it could also be *Incorrect*. This can happen in scenarios not covered by the tests. For example, an overfitting patch may produce an unexpected output for an input not included in the test cases, that is, it *overfits* the available tests and breaks untested but desired functionality [59].

To show the real effectiveness of an APR, researchers introducing new repair approaches have reported the amount of both plausible and correct generated patches. To illustrate the magnitude of the problem, the study by Liu et al. [19] shows that TBar, one of the most effective APR tools, produced 24 correct patches out of 72 plausible ones: 66.6% of the generated patches are incorrect.

Initial analyses of overfitting were performed by humans by annotating patches as either overfitting or correct (e.g [19, 60]). Additionally, another type of analysis generates additional test cases not included in the buggy program, thus not used for validating candidate patches (e.g. [59, 63]). Then, different patch overfitting detection systems have emerged, such as PatchSim [64] and DiffTGen [65], xTestCluster [66], all of these based on test case generation. The main idea of these approaches is to generate new inputs that allow to differentiate the behaviors of overfitting patches from correct ones. Wang et al. [67] conducted a preliminary empirical study on the effectiveness of automated patch overfitting detection, while Petke et al. [68] studied the magnitude of the overfitting problem.

Just Accepted

Then, researchers have trained ML models using patches labeled from previews work. These ML-based approaches, given a plausible patch as input, produce a binary decision (correct or overfitting) and/or return an overfitting score. Some of these ML-based approaches are ODS [69], Shibboleth [70], CACHE [71], and Tian et al. [72].

At the same time, repair techniques have incorporated mechanisms to avoid overfitting patches, such as Prophet [45], ObjSim [73] and CapGen [41] which incorporate patch ranking module (in the case of Prophet trained from labeled patches) to rank correct patches at high positions before incorrect plausible ones.

In this paper, we study the impact of overfitting patches on the energy consumed by an APR tool.

2.2 Energy consumption of software

Energy consumption has been investigated in the scope of different types of systems, such as mobile apps [74], software product lines [75] and more recently, ML-based systems [76]. As a matter of example, Strubell et al. quantified the approximate environmental costs of training a variety of successful neural network models [77]. They estimated that a human life per average implies 11,023 CO_2 emissions in one whole year whereas training one big Transformer model with neural architecture search emits 626,155 CO_2. In other work, Schwartz et al. reported that the cost of training ML models (in terms of the amount of computing resources used) as required by state-of-the-art techniques has increased by a factor of 300.000x in only 6 years, doubling every few months [18]. This increase is primarily due to modeling and algorithmic choices, over hardware considerations [18], which has motivated several researchers to focus on software when searching for energy consumption reduction [76, 78].

Hort et al. presented a literature study on the energy use of language models for source code [79]. Among different software engineering tasks, they studied papers on program repair and report that just four works on that field ([80], [48], [81] and [82]) provide details about *a)* hardware use for training the model, and *b)* time and energy spend for training. More recently, Shi et al. presented a roadmap on LLMs for software engineering tasks [83]. However, none of them focus on the energy consumption of the repair process (i.e., the inference and the validation of patches), as we do in this paper.

Moving towards the ecological transition, software development cannot only target accuracy, but energy efficiency as well. In this context, both traditional and state-of-the-art neural network-based (including LLM-based) APR tools shall consider the optimizations of their parameters and the amounts of energy and the resources used. However, this is not easy because software developers lack critical information and knowledge about software energy consumption [84, 85].

3 METHODOLOGY

In this paper, the type of investigation we choose to carry out the empirical study is *Technology-oriented Experiment* [86]. This presents the method applied to carry out the experiment that allows us to answer the research questions, presented in Section 3.1.1. Figure 1 shows an overview of the experiment process. Each gray box corresponds to an activity explained in the remaining subsections.

3.1 Experiment Scope

3.1.1 Research Questions. The research goal defined in Section 1 breaks down in the following research questions (RQs) about energy consumption of ARP tools:

(1) *What is the energy consumption of APR tools on Defects4J?.*
This RQ aims to obtain the energy profile of APR tools in the task of repairing real bugs.
(2) *Which APR tool achieves the best energy efficiency in correctly fixing bugs*
This RQ aims to investigate the trade-off between energy consumption and the repairability of each tool.

Just Accepted

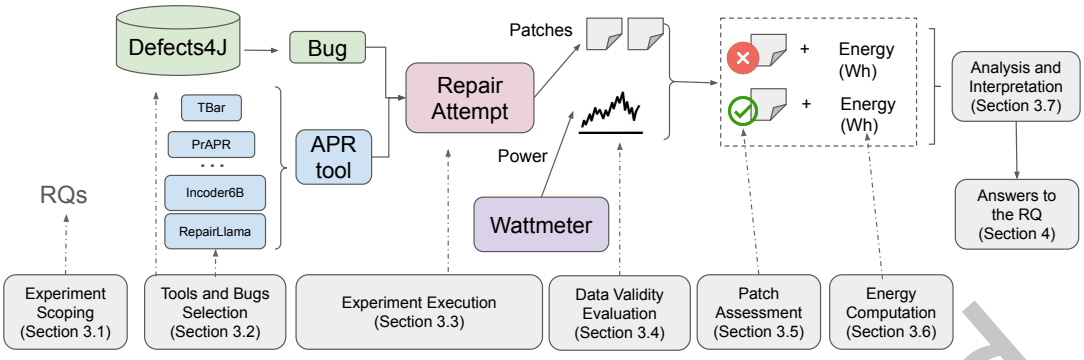


Fig. 1. Experiment process for measuring energy consumption of APR tools.

(3) *What is the impact of patch overfitting on energy consumption of APR?* This RQ aims to measure the amount of energy that is destined for a successful repair and the amount that is wasted in unsuccessful repairs with overfitting patches.
(4) To what extent does the energy consumed to find a patch correlate with: a) the repair time?, b) difficulty to fix a bug, c) the number of patch candidates (NPC) and d) project under repair?
This RQ checks whether there is a correlation between energy and other metrics that represent the repair time, efficiency, difficulty to repair and project under repair.

3.1.2 Variables. Our experiment has two independent variables: *1)* Bug, and *2)* APR tool; and the following dependent variables: *a)* number of bugs with plausible patches, *b)* number of bugs with correct patches, *c)* time and energy spent to find the first plausible patch, *d)* time and energy spent to find the first correct patch, *e)* energy spent during a repair attempt.

3.2 APR Tools and Bugs Selection

In this section we define the criteria to select the APR tools and bugs of our experiment. We decide to study the energy consumption of tools capable of repairing Java bugs. For this reason, we choose the benchmark named Defects4J [11], which is a benchmark of Java written buggy programs, commonly used to evaluate the repairability of Java bugs (e.g., [19, 87]. We focus on the version 1.2 a benchmark composed of 395 Java bugs. Each of these bugs is exposed by at least one test case. A *repair attempt* is the execution of a repair tool (e.g., TBar) on a particular bug from Defect4J (e.g., Math-70) under configuration given to the tool as input. With respect to the APR tools, we focus on two families of tools that have produced state-of-the art results over the last 10 years: *1) traditional repair tools* (inc. search-based, constraint-based and template-based), and *2) LLM based repair tools*. We now describe the selection process for the tools of each family.

Traditional repair tools. We apply the same criteria as Durieux et al. [87] and Liu et al. [19], which include: *1)* availability of the tool, *2)* availability of the source code of the tool (e.g., on a code platform such as Github or Gitlab), *3)* the tool receives as input the buggy program to be repaired, but does not receive any additional information about the bug.

Following the above criteria, we select in this study the following traditional tools: Avatar [88], FixMiner [43], Nopol [47], SimFix [28], TBar [27], Dynamoth [89], jGenProg [57] (implementation of GenProg [8] for Java bugs), jMutRepair [60], and kPAR [40] (implementation by [90]). As suggested by their authors, SimFix [28] is used instead of the ACS tool [91]. We also add PRarp [24], which was not considered in [19, 87]. We discard repair tools

Just Accepted

implemented in the ARJA framework (which include ARJA, KALI-A, GenProg-A and RSRepair, among others) because, after inspecting their patches from [87], we found a large amount of non-plausible ones. Furthermore, given the limited computational resources we have to execute our large-scale experiment (see Section 3.3.3), we discard Java implementations of the Kali approach [61] and Cardumen [92] because, as observed by [19, 87], they have a lower ability to find correct patches.

Large language model (LLM)-based repair tools. A recent new family of APR approaches is based on neural networks, whose ability to repair Java bugs has been shown in a number of studies (e.g., [20, 48–52, 93, 94]). The pioneer approaches of this family trained the networks from scratch, using a large dataset of real bug fixing (e.g., [48]). More recently, researchers proposed new approaches based on pre-trained LLMs (e.g., [31, 53]) or chatbots (e.g., [54, 95]).

In this study, we represent the family of neural network-based approaches with the tools proposed by two works. First, we choose the work from Jiang et al. [31] for the following reasons:

- Provides usable tools that allow to reproduce their experiment.
- Uses ten open-source code LLMs from four families of large models: InCoder from Meta [29], CodeGen and CodeT5 [96] from SalesForce [30], and PLBART [97].
- Fine-tunes these 10 models using a dataset of 143,666 bug fixes, provided by Zhu et al. [50]. These fine-tuned models are also publicly available.
- The fine-tuned models achieve state-of-the-art results, improving the performance of other neural network-based approaches, including RewardRepair [20], CURE [94], RECODER [50] and KNOD [52].

The ten LLMs used by Jiang et al. are: *1)* CodeGen-350m (350 millions parameters), *2)* CodeGen-2B (2 billions), *3)* CodeGen-6B (6 billions), *4)* CodeT5-small (60 millions), *5)* CodeT5-base (220 millions), *6)* CodeT5-large (770 millions), *7)* InCoder-1B (6.7 billions), *8)* InCoder-6B (1.3 billions), *9)* PLBART-Base (140 millions), *10)* PLBART-Large (400 millions).

Secondly, we use RepairLlama from Silva et al. [32] for the following reasons:

- RepairLlama is built on top of CodeLlama [33] a state of the art LLM for code related task. CodeLlama is not included in the work from Jiang et al. as it was released after their work.
- Applies the state-of-the-art fine-tuning technique named PEFT (parameter-efficient fine-tuning) for program repair. Instead of fine-tuning the entire network as done by Jiang et al., PEFT means that Silva et al. just fine-tune a small portion of the network, the adapters.

In this work, we use the RepairLlama version `IR4 x OR2` which is the one that achieves the highest number of correct patches according to [32], and is built on top CodeLlama7B (7 billion parameters).

3.3 Experiment Execution

3.3.1 Tools preparation. Before launching the experiment, we need to modify some of the tools. By default, publicly available implementations of TBar, Avatar, FixMiner, and kPar, stop when the first plausible patch is found or after trying a fixed number of candidate patches (e.g., 10.000 for TBar). As we are interested in computing the energy to repair *correctly* a bug, and the first patch found can be plausible but incorrect, we modified the source code of these tools with the goal of continuing the search after the first patch found, until the execution reaches a timeout. For the other traditional tools (e.g., Nopol, jGenProg), we indicate via the provided input arguments to continue the search.

We also modify all traditional and LLM-based tools considered in this study to register the timestamp corresponding to the instant a plausible patch is found. This moment corresponds to the end of the validation of a patch. This timestamp allows us to compute the energy used from the beginning of the repair attempt to the

Just Accepted

moment the patch is found and validated. We also modify the code to register the number of candidate patches synthesized (e.g., those that do not compile) in order to compute the metric NPC [25].[2]

3.3.2 Experiment setup. In order to execute LLM-based tools, we use the scripts provided by Jiang et al. and from Silva et al. to repair Defect4J bugs using the fine-tuned version of these models, also provided by the authors[3]. We do not change any parameter or hyperparameter. By default, their script queries the LLMs models to generate ten candidate patches per buggy line. For each of these patches, the script compiles it, applies it to the buggy project, then executes the initially failing test cases, and finally executes the test suite provided by the buggy program. We modify their scripts to register the timestamps corresponding to the start and end of each of these phases.

Experiments on traditional APRs (e.g., [19]) usually have different setups than experiments on LLM-based APRs (including Jiang et al. [31] and Silva et al.[32]). Two of the main differences are the following. First, the considered LLM-based tools do not execute the fault localization phase; they give as input to the model only the buggy location. That is known as *perfect fault localization* [19]. In the case of Jiang et al. the input is the buggy line (i.e., the line where the human-written patch is located) and for Silva et al. is the buggy function. In contrast, the traditional tools studied in this paper do not require the buggy line(s) as input. Consequently, they apply a fault localization phase in order to discover potential locations on the code (aka suspicious code) where candidate patches could be applied. This setup is known as *Normal Fault Localization* [19] Second, Jiang et al. focus on a subset of bugs, namely single-hunk bugs, because, as they mention, the best deep learning-based APR techniques are all specially designed for Java single-hunk [31]. Note that RepairLlama from Silva et al. is able to repair multi-hunk bugs, but these hunks must be located in the same function. For these reasons, to avoid unfair comparison, we analyze separately the energy consumption of traditional approaches and LLM-based and traditional approaches, not aiming to compare the results between these two families of approaches.

3.3.3 Execution platform. We perform repair attempts on a high-performance computing (HPC) cluster composed of two families of identical nodes: *1)* nodes with CPU (Intel Xeon Gold 5220, 96 GB RAM); *2)* nodes with CPU and GPU (Intel Xeon E5-2698 v4, 512GB RAM, 8 GPUs Tesla V100-SXM2-32GB). Given the execution scripts from [31], GPU-equipped nodes are required to run the selected LLM models.

Each repair attempt is executed on a dedicated node. No other user has access to the nodes assigned to this study, and we do not execute any other task in parallel. Before the execution of a repair attempt, the cluster deploys an image with the default standard environment (OS + libraries). This image is the same for every repair attempt. It is important to note that the environment does not execute any automated update after the image deployment (e.g. library update from the internet). All of this ensures that all repair attempts are executed under the same conditions. Each node is individually monitored by an OmegaWatt vendor wattmeter[4], which provides the electrical power consumed each 20 milliseconds with a precision of 0.1 watt. An experiment based on energy measurement should be performed several times in order to remove energy measurements that are potentially outliers [98]. For this reason, we execute each repair attempt five times and then compute the median of the metrics obtained on each repair attempt.

In a repair attempt, before and after a repair tool execution, we execute the command `sleep` with 30 second as parameter, in order to capture the power of the machine in idle state. That helps us to detect eventual anomalies, as we explain later.

For the execution of traditional APR tools, we set a timeout of 5 hours for each repair attempt, which is larger than those used in previous Java repair experiments [60, 87]. The repair attempts are executed on the nodes having just CPUs, as GPU is not required by traditional APR tools.

[2]We need to compute NPC ourselves because in Liu et al. [19], the work that uses NCP as a proxy of efficiency, the NPC per repair attempt (e.g., tool-bug pair) is not shown, they only report a summary per tool.
[3]Replicability packages from [31]: https://github.com/lin-tan/clm and from [32]: https://github.com/ASSERT-KTH/repairllama
[4]https://mv.omegawatt.fr/ (Wattmetter model under request to the HPC support.)

Just Accepted

LLM-based APR usually has, at least, two phases: *1) Patch inference*, where the model generates n candidate patches (where n corresponds to the beam size). This phase usually requires a node with GPUs, according to the model's requirements; *2) Patch validation*, where the APR approach compiles and validates candidate patches. This phase requires just CPUs.[5] Then, we sum the energy consumed by both phases. Note that the LLMs have different requirements on GPUs: InCoder-2B and CodeGen-2B use 4 GPUs, CodeLlama, InCoder-6B and CodeGen-6B use 8 GPUs, and the others use 1 GPU.

3.4 Data Validity Evaluation

A repair attempt produces (for a given tool and bug) has as output: (1) a list of *plausible* patches, and (2) power samples. Each patch and sample are associated with a timestamp. Once executed, we perform two validations. First, we (re)check that all the plausible patches compile and pass all test cases. Second, we verify the absence of anomalies on the power consumption at the beginning and end of a repair attempt. An anomaly could be caused by the execution of an unexpected process. For detecting them, we retrieve the power samples from the idle interval, executed at the beginning and end of the attempt with the sleep command. We mark as valid those attempts for which these samples are ≈62 W. This value corresponds to the power consumption of the node in idle mode, and was precomputed by us before launching the experiment.

3.5 Patch Assessment

To determine the correctness of the plausible patches found during our study, we reuse the patch assessment done by Liu et al. [19] and CACHE [71], which manually labeled each generated patch for Defects4J bugs as *correct* or *plausible* (i.e., incorrect). From these sets of labeled patches, we labeled the patches we generated in our experiment as follows. For each patch P that an approach generated to fix a bug B in our experiment, we check whether P exists on the mentioned sets of labeled patches. For that, we inspect all the candidate patches that fixed B. We search for a patch P' that are syntactically or semantically equivalent to P. If P' exists, we reuse its label. Otherwise, if we cannot find P' there, it means that the patch was not previously analyzed in the mentioned works [19][71]. In that case, we perform a manual assessment of P by comparing it against the ground-truth patch (*aka* golden patch), available on the Defects4J framework. We conduct the analysis by following the guide provided by Liu et al [19].

3.6 Energy Computation

To respond to the research questions, we compute the following three metrics: E_{ra}, E_{fp} and E_{fc}, which are three measures of energy. Figure 2 illustrates the correspondence between these metrics and the repair process, in particular, with two repair attempts. At the top (a), the repair attempt produces two patches, the first one plausible but incorrect, the second one is correct. In the bottom (b), the repair attempt does not produce any patch.

- E_{ra}: the energy consumed during a **repair attempt** (i.e., one tool repairing one bug) until it stops (either because the search has finished or the timeout is reached).
- E_{fp}: the energy spent by an approach from the beginning of a repair attempt until the **first plausible** patch is found and validated. The moment that patch is found is t_{fp}.
- E_{fc}: the energy spent by an approach from the beginning of a repair attempt until the **first correct** patch is found and validated. The moment that patch is found is t_{fc}.

[5]It would be possible to run both phases in a node with GPU, however, it is an expensive strategy: the validation of patches -which can take up to dozens of seconds- does not require GPU, thus running it on this type of node would waste GPU time, much expensive than GPU time.

Just Accepted

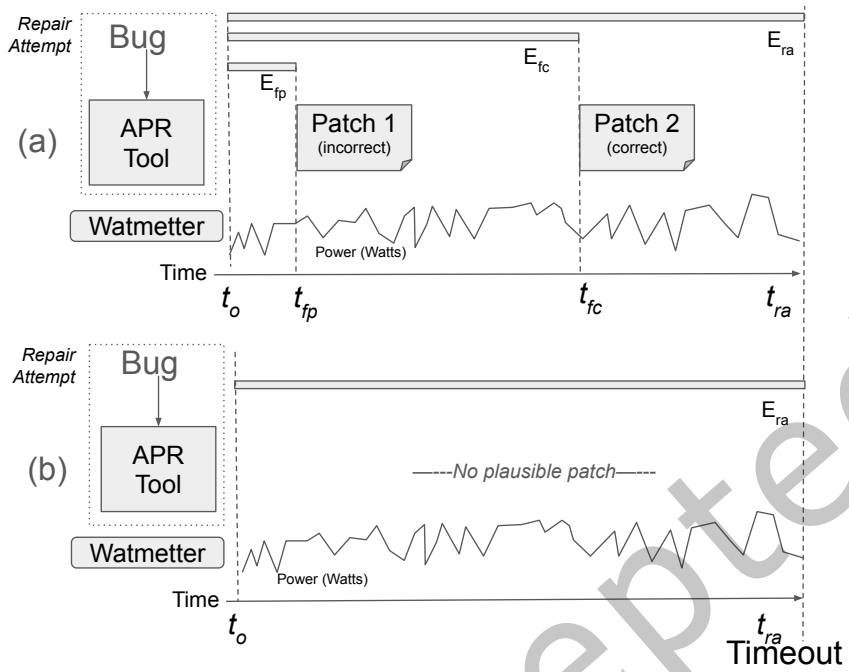


Fig. 2. Energy metrics from two repair attempts a) with two plausible patches: the first one incorrect (*overfitting*), the second one correct, b) with no patches.

3.6.1 Computing the energy metrics from repair attempts. We compute the energy metrics E_{ra}, E_{fp} and E_{fc} using the following formula.

$$E = \int_{t_0}^{t_x} Pw(t) \tag{1}$$

The energy is calculated as an integral of all the power values captured between times t_0 and t_x. In particular, t_0 is the start of the repair attempt and t_n is the timestamp associated with different events. For the E_{ra} metric, t_x is the timestamp corresponding to the end of a repair attempt. For the E_{fp} metric, t_x is the timestamp corresponding to the first plausible patch found. For the E_{fp} metric, t_x is the timestamp corresponding to the first correct patch found.

In practice, we compute the Equation 1 as follows.

$$E = \int_{t_0}^{t_x} Pw(t) \approx \frac{(t_x - t_0)}{2}[p_0 + 2 * p_1 + \cdots + 2 * p_{x-1} + p_x] \tag{2}$$

where $p_0, p_1, \ldots, p_x$ are the power values captured between t_0 and t_x. We report the energy using the Watt-hour (Wh) unit.

3.7 Experiment Analysis and Interpretation

In this section, we detail the steps that are taken to respond to each research question.

3.7.1 RQ 1: What is the energy consumption of APR tools on Defects4J?. We summarize the energy consumed by a particular tool in all repair attempts made on Defects4J bugs using the following formula.

Just Accepted

$$Total_Energy(\text{tool}, \text{bugs}) = \sum_{i=1}^{|bugs|} \begin{cases} E_{fp} & \text{if tool finds plausible patch for bug } i \\ E_{ra} & \text{otherwise} \end{cases} \tag{3}$$

This equation uses the metrics E_{ra} and E_{fp}, explained in Section 3.6. As in this research question, we reproduce the experiment from [19] for traditional APRs, which stops after the first plausible patch, Eq. 3 considers the energy until that point (i.e., E_{fp}). For LLM-based, we reproduce the experiments from [31] and [32] which both use perfect fault localization and generate ten candidate patches per bug. Similarly, Eq. 3 considers the energy consumed until the first plausible is found.

Comparison of energy between APRs. We also conduct an comparison of energy consumption between approaches. In particular, we test if there is a significant difference on the energy consumed by two approaches. Our null hypothesis H_0 states that *there is no significant difference between the energy consumed by two given approaches.* The alternative hypothesis H_a states that *there is significant difference between the energy consumed by two given approaches.* We test H_0 using Mann-Whitney test.

Energy on Inference Phase on LLM-based APRs. As mentioned in Section 3.3.3, LLM-based APRs execute two main phases: inference and patch validation. We compute the energy spent by each one as follows. In each phase execution, we register two timestamps: at the beginning and at the end of the phase. Then, we retrieve from the wattmetter the power samples between the two timestampts. Finally, we compute the energy of each phase using the formula described in Equation 2.

Comparison of energy consumptions from a traditional APR to LLM-based APRs. As mentioned previously, the setups from traditional and LLMs-based experiments are different. This prevents us from doing a fair comparison between them. To make a fair comparison, we choose one traditional APR tool, named TBar [27], to represent this family of approaches and compare it with the LLM-based APRs. We choose it because it is the traditional tool that most bugs from Defects4J correctly fixes [19]. Then, we create a modified version of TBar, called TBar10C, with a setup similar to the one used by the LLM-based experiment: *1)* TBar10C uses perfect fault localization, and *2)* TBar10C generates up to 10 candidate patches (which are not necessary plausible).

3.7.2 RQ 2: Which APR tool achieves the best energy efficiency in correctly fixing bugs. To respond to this research question, we present $E/\#C$ metric, which reports the energy spent per correct bug. As Eq.4 shows, it divides the total energy that a tool spent on the whole study (obtained from Eq. 3) by the total number of first plausible patches found by that tool marked as correct.

$$E/\#C(tool, bugs) = \frac{Total_Energy(tool, bugs)}{\#\text{ Correct first patches found by tool}} \tag{4}$$

3.7.3 RQ 3: What is the impact of patch overfitting on energy consumption of APR?. Unlike previous research questions, where traditional approaches stopped after the first plausible patch (which could be correct or incorrect) following [19], in this RQ our experiment continues the search after it has been found and until it reaches the timeout or finishes the search (that is, it fully scans the search space). With this new setup, we let traditional APR tools continue the search in order to find a correct patch after a plausible (and incorrect) one.

For each repair tool, we report three measures. First, $Energy_{Correct}$, the sum of the energy consumed by the tool to find the first correct patch in all Defects4J.

$$Energy_{Correct}(\text{tool}, \text{bugs}) = \sum_{i=1}^{|bugs|} \begin{cases} E_{fc} & \text{if tool repairs bug } i \\ 0 & \text{otherwise} \end{cases} \tag{5}$$

Just Accepted

This equation uses the metric E_{fc}, introduced in Section 3.6.

Secondly, $Energy_{Overfitting}$, the sum of the energy consumed by a tool in all repair attempts that produce only overfitting patches.

$$Energy_{Overfitting}(\text{tool}, \text{bugs}) = \sum_{i=1}^{|bugs|} \begin{cases} E_{ra} & \text{if tool just generates overfitting patches for bug } i \\ 0 & \text{otherwise} \end{cases} \tag{6}$$

This equation uses the metric E_{ra}, explained in Section 3.6.

Lastly, $Energy_{Unrepaired}$, the sum of the energy consumed by a tool in all repair attempts that do not produce any plausible patch.

$$Energy_{Unrepaired}(\text{tool}, \text{bugs}) = \sum_{i=1}^{|bugs|} \begin{cases} E_{ra} & \text{if tool does not generate any plausible patch for bug } i \\ 0 & \text{otherwise} \end{cases} \tag{7}$$

3.7.4 RQ 4: To what extent does the energy consumed to find a patch correlate with: a) the repair time?, b) difficulty to fix a bug, c) the number of patch candidates (NPC) and d) project under repair? To answer this research question, we compute the Spearman correlation between the energy and a set of metrics. First, we compute the correlation between the energy consumed to find a patch (E_{fp}) and the time required to find it (t_{fp}), and then similarly between E_{fp} and the Number of Patch Candidates NPC_{fp}, which was used as a proxy of the APR efficiency by [19]. In our experiment, we register the NPC metric as explained in Section 3.3.

Then, we compute the correlation between the energy for a correct patch E_{fc} and the difficulty for repairing it. As a proxy of difficulty, we use two of the parameters used by Motwani et al. [26] to study the relationship between the ability of approaches to repair hard and important bugs:

1) Time-to-fix ($T2F$): the difference, in days, between when the bug's issue was reported and when that issue was resolved, and
2) Priority (Pr): the priority assigned to the defect's issue in the bug tracking system. Motwani et al. mapped these values on a scale of 1 (lowest) to 5 (higher).

Finally, we analyze the differences in energy consumption according to the repair project in question. We first analyze the energy that each test suite consumes (we recall that test suites are used to validate candidate patches). Then, we discuss the energy required to repair a patch according to the projects.

4 RESULTS

4.1 RQ1: What is the energy consumption of APR tools on Defects4J?

4.1.1 Energy consumption of traditional APR. Table 1 shows the energy consumed by traditional APRs approaches in Defects4J, represented visually in Figure 3. In the table, `# Plausible Patch` shows the total number of bugs with at least one patch, while `# Correct Patch` shows the number of bugs correctly repaired with the first plausible patch. We recall that, to answer this RQ, we reproduce the experiment from [19] which stops the patch search after the first plausible patch found. Column $Total_Energy$ shows the sum of all the energy consumed by the tool on all bugs.

First, we observe important differences in the energy consumed by the approaches. In particular, three of them (TBar, jGenProg and Avatar) consumed much more energy than the others. The amount of energy consumed decreases for the rest of the approaches, and remarkably two of them, Parpr and jMutRepair, consumed much less energy (less than 10 Wh). consumeing more energy does not mean being able to correctly repair more bugs. In other words, there is no (positive) correlation between energy and repairability. Figure 3 shows this trade-off: The green line (repairability) does not follow the bars (energy).

Just Accepted

Approach	Patches #Plausible	Patches #Correct	*Total_Energy* (Wh)	*E/#C*
Avatar	45	15	110,775	7,385
FixMiner	26	9	38,344	4,260
Nopol	91	1	17,911	17,911
PraPR	75	11	3,389	308
SimFix	52	21	72,576	3,456
TBar	71	28	118,558	4,234
dynamoth	61	1	37,752	37,752
jGenProg	40	5	111,810	22,362
jMutRepair	32	3	7,151	2,383
kPAR	40	9	73,615	8,179
TBar10C	39	31	767	24.7
RepairLlama	77	61	1,263	20.7
CodeGen-350M	27	24	594	24.7
CodeGen-2B	39	30	756	25.2
CodeGen-6B	45	37	837	22.6
CodeT5-small	23	18	572	31.7
CodeT5-base	34	28	564	20.1
CodeT5-large	42	31	608	19.6
InCoder-1B	44	27	737	27.3
InCoder-6B	57	37	820	22.2
PLBART-base	34	24	627	26.1
PLBART-large	41	30	567	18.9

Table 1. Energy consumed by APRs approaches in Defect4J to find the first plausible patch.

Just Accepted

TBar is the most consuming approach and, at the same time, the most effective: 28 of the first patches found are correct. We actually observe than the approaches that most energy consumes are based on templates[6]. Such approaches can have large search spaces: Since a template needs to be initialized with local and global variables, buggy locations with many variables in scope can result in a large number of candidate patches that need to be compiled and, eventually, validated using test cases. The particularity of TBar, which may explain the better effectiveness w.r.t. to the other, is that its search space is formed by templates proposed by different previous works. That is, it implements a curated list of repair templates. However, this effectiveness comes at the expense of energy (118,558 Wh consumed).

The second most effective approach is SimFix, which correctly fixed 21 bugs with the first plausible patch. However, it consumed much less energy than TBar and other approaches: It is in the fifth place with 72,572 Wh. We suspect that the reason for this *good* trade-off between energy and repairability is that SimFix creates a tailored search space for each buggy program by intersecting one search space mined from previous patches and another search space created from similar changes done in the project under repair. This strategy helps Simfix to consider templates that yield correct repairs and discard those that may not be useful to fix the bug under repair, saving energy required for patch compilation and test execution.

[6]jGenProg does not have a list of pre-defined templates, but it creates a similar (known as *ingredients*) extracted from the program under repaired which are then instantiated to the buggy location.

	Avatar	FixMiner	Nopol	PraPR	SimFix	TBar	dynamoth	jGenProg	jMutRepair	kPAR
Avatar	-	0.00 (-0.56 L)	0.00 (-0.63 L)	0.00 (-0.90 L)	0.00 (-0.26 S)	0.02 (0.10 N)	0.00 (-0.49 L)	0.30	0.00 (-0.79 L)	0.00 (-0.22 S)
FixMiner	0.00 (0.56 L)	-	0.16	0.00 (-0.52 L)	0.00 (0.42 M)	0.00 (0.61 L)	0.15	0.00 (0.56 L)	0.00 (-0.25 S)	0.00 (0.33 S)
Nopol	0.00 (0.63 L)	0.16	-	0.00 (-0.53 L)	0.00 (0.53 L)	0.00 (0.67 L)	0.01 (0.12 N)	0.00 (0.63 L)	0.00 (-0.21 S)	0.00 (0.36 M)
PraPR	0.00 (0.90 L)	0.00 (0.52 L)	0.00 (0.53 L)	-	0.00 (0.91 L)	0.00 (0.92 L)	0.00 (0.62 L)	0.00 (0.89 L)	0.00 (0.44 M)	0.00 (0.86 L)
SimFix	0.00 (0.26 S)	0.00 (-0.42 M)	0.00 (-0.53 L)	0.00 (-0.91 L)	-	0.00 (0.34 M)	0.00 (-0.36 M)	0.00 (0.29 S)	0.00 (-0.77 L)	0.60
TBar	0.02 (-0.10 N)	0.00 (-0.61 L)	0.00 (-0.67 L)	0.00 (-0.92 L)	0.00 (-0.34 M)	-	0.00 (-0.54 L)	0.76	0.00 (-0.82 L)	0.00 (-0.29 S)
dynamoth	0.00 (0.49 L)	0.15	0.01 (-0.12 N)	0.00 (-0.62 L)	0.00 (0.36 M)	0.00 (0.54 L)	-	0.00 (0.51 L)	0.00 (-0.32 S)	0.00 (0.24 S)
jGenProg	0.30	0.00 (-0.56 L)	0.00 (-0.63 L)	0.00 (-0.89 L)	0.00 (-0.29 S)	0.76	0.00 (-0.51 L)	-	0.00 (-0.78 L)	0.00 (-0.25 S)
jMutRepair	0.00 (0.79 L)	0.00 (0.25 S)	0.00 (0.21 S)	0.00 (-0.44 M)	0.00 (0.77 L)	0.00 (0.82 L)	0.00 (0.32 S)	0.00 (0.78 L)	-	0.00 (0.51 L)
kPAR	0.00 (0.22 S)	0.00 (-0.33 S)	0.00 (-0.36 M)	0.00 (-0.86 L)	0.60	0.00 (0.29 S)	0.00 (-0.24 S)	0.00 (0.25 S)	0.00 (-0.51 L)	-

(a) Traditional APRs

	RepairLlama	TBar10C	CodeGen-2B	CodeGen-350M	CodeGen-6B	CodeT5-base	CodeT5-large	CodeT5-small	InCoder-1B	InCoder-6B	PLBART-base	PLBART-large
RepairLlama	-	0.00 (-0.4 M)	0.02 (-0.2 S)	0.00 (-0.4 M)	0.25	0.00 (-0.4 M)	0.00 (-0.3 S)	0.00 (-0.3 M)	0.00 (-0.3 S)	0.01 (-0.2 S)	0.00 (-0.3 M)	0.00 (-0.4 M)
TBar10C	0.00 (0.4 M)	-	0.00 (0.3 S)	0.27	0.00 (0.3 M)	0.15	0.05	0.24	0.02 (0.2 S)	0.00 (0.3 S)	0.12	0.53
CodeGen-2B	0.02 (0.2 S)	0.00 (-0.3 S)	-	0.00 (-0.3 S)	0.12	0.00 (-0.3 S)	0.01 (-0.2 S)	0.00 (-0.2 S)	0.02 (-0.2 S)	0.98	0.00 (-0.2 S)	0.00 (-0.3 S)
CodeGen-350M	0.00 (0.4 M)	0.27	0.00 (0.3 S)	-	0.00 (0.4 M)	0.66	0.28	0.87	0.12	0.00 (0.3 S)	0.58	0.58
CodeGen-6B	0.25	0.00 (-0.3 M)	0.12	0.00 (-0.4 M)	-	0.00 (-0.3 M)	0.00 (-0.3 S)	0.00 (-0.3 S)	0.00 (-0.3 S)	0.12	0.00 (-0.3 S)	0.00 (-0.4 M)
CodeT5-base	0.00 (0.4 M)	0.15	0.00 (0.3 S)	0.66	0.00 (0.3 M)	-	0.46	0.82	0.28	0.00 (0.2 S)	0.89	0.32
CodeTt-large	0.00 (0.3 S)	0.05	0.01 (0.2 S)	0.28	0.00 (0.3 S)	0.46	-	0.41	0.73	0.02 (0.2 S)	0.61	0.11
CodeT5-small	0.00 (0.3 M)	0.24	0.00 (0.2 S)	0.87	0.00 (0.3 S)	0.82	0.41	-	0.22	0.00 (0.2 S)	0.77	0.42
InCoder-1B	0.00 (0.3 S)	0.02 (-0.2 S)	0.02 (0.2 S)	0.12	0.00 (0.3 S)	0.28	0.73	0.22	-	0.03 (0.2 S)	0.35	0.04 (-0.1 S)
InCoder-6B	0.01 (0.2 S)	0.00 (-0.3 S)	0.98	0.00 (-0.3 S)	0.12	0.00 (-0.2 S)	0.02 (-0.2 S)	0.00 (-0.2 S)	0.03 (-0.2 S)	-	0.00 (-0.2 S)	0.00 (-0.3 S)
PLBART-base	0.00 (0.3 M)	0.12	0.00 (0.2 S)	0.58	0.00 (0.3 S)	0.89	0.61	0.77	0.35	0.00 (0.2 S)	-	0.31
PLBART-large	0.00 (0.4 M)	0.53	0.00 (0.3 S)	0.58	0.00 (0.4 M)	0.32	0.11	0.42	0.04 (0.1 S)	0.00 (0.3 S)	0.31	-

(b) LLM-based APRs

Table 2. Comparison between approaches using the Mann-Whitney test. Cells show $p-values$, those with $p-values < 0.05$ (that is, rejection of H_0) are colored black. The Cliff's Delta is shown in parentheses, along with its interpretation (Negligible, Small, Medium, Large).

Just Accepted

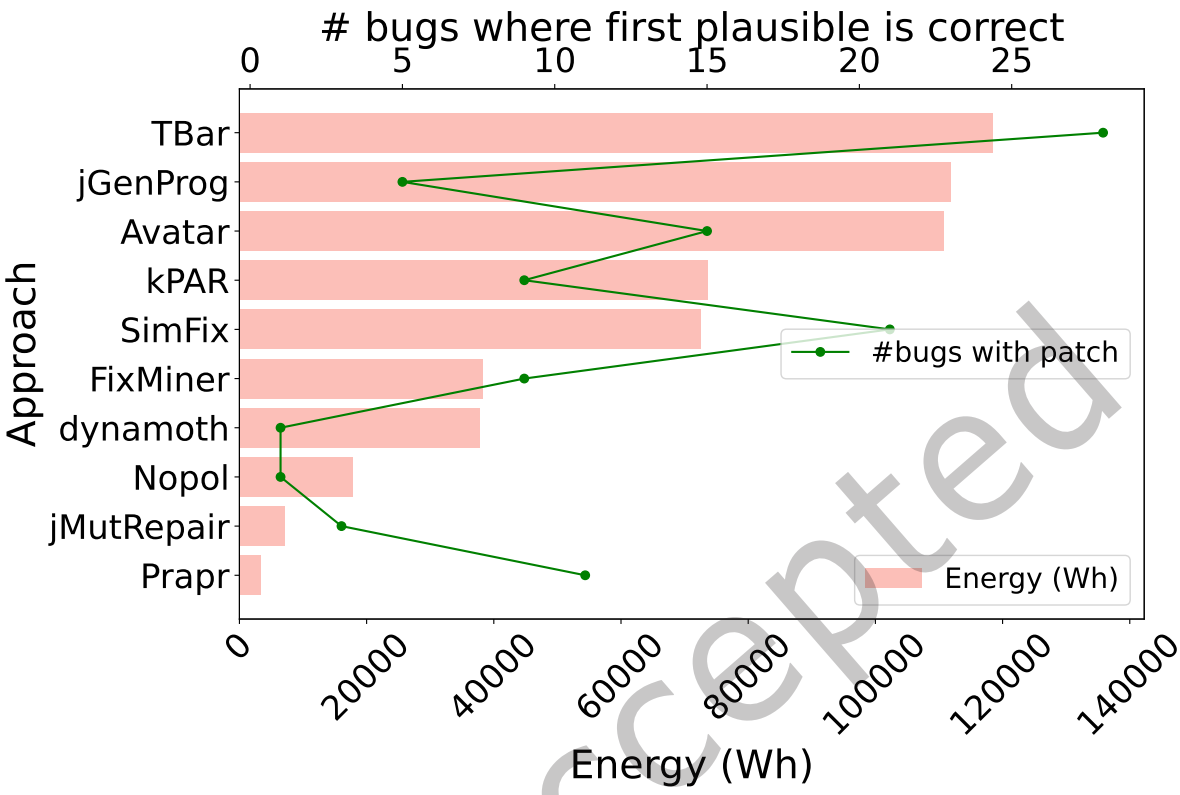


Fig. 3. Energy consumed by *traditional* APR approaches in Defect4J to find the plausible patch.

Just Accepted

Let us focus on those that consumed less energy. PRapr was designed to be scalable [99]: It does not apply symbolic execution or constraint solving, and it applies bytecode-level manipulation. We conjecture that this explains its low energy consumption w.r.t. the others. Even so, PraPR correctly fixed 11 bugs with the first plausible patch. Both low-energy and repairability from Nopol and jMutRepair (17,911 and 7,151 Wh, respectively) can be explained by the fact that they focus on particular defects classes (e.g., Nopol focuses on two: missing preconditions and buggy Ifs) and, by consequence, their search space may be smaller. The Spearman correlation between between energy and number of bugs correctly repair gives a coefficient of 0.5 (p-value=0.1411), which indicates a *moderate correlation*.

Table 2 presents the comparison of energy consumption between approaches using Mann-Whitney test. The cells show the p-values from the test. Black cells (where $p-values < 0.05$) indicate rejection of the null hypothesis H_0 (which asserts that there is no significant difference between the two approaches) at a significance level of 0.05. For most of the APR pairs, H_0 is rejected, which means that there are significant differences on the energy consumption. For few comparisons H_0 cannot be rejected, e.g., kPAR and SimFix consume similar energy (as also shown in Fig. 3). Moreover, we compute the *effect size*, which quantifies the strength or magnitude of the relationship or difference between two variables. This is measured using Cliff's Delta, presented in parentheses for each cell, along with its interpretation (Negligible, Small, Medium, and Large effect). We observe that the

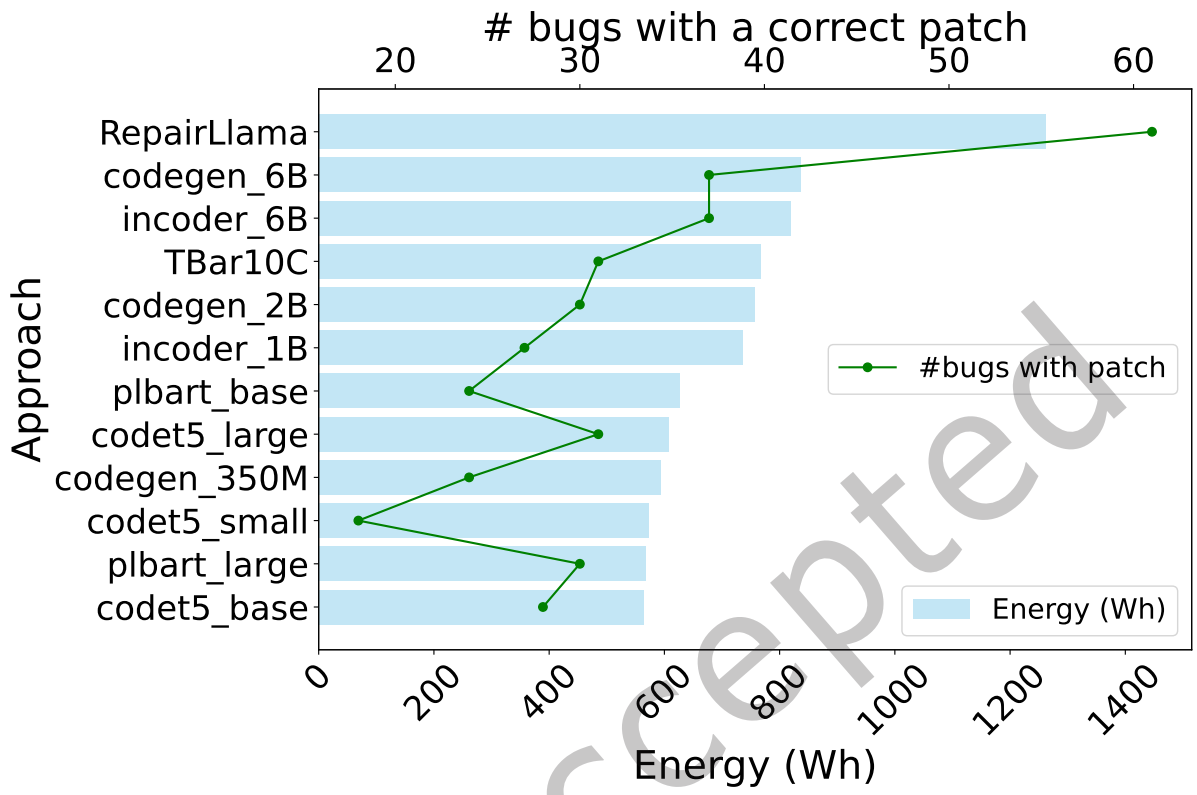


Fig. 4. Energy consumed by *traditional* APR approaches in Defect4J to find the plausible patch.

Just Accepted

effect size is Large for most comparisons with a significant difference (46 cases), while the remaining are Medium (12 cases), Small (18 cases), and Negligible (4 cases).

4.1.2 Energy consumption of LLM-based repair tools. Table 1 shows the energy consumed by LLM-based APRs in Defects4J, represented visually in Figure 4. Even TBar10C (introduced in Section 3.7) is shown in the tables and figures along this section, for the moment please ignore it, we discuss it in Section 4.1.3.

The energy consumption of LLM-based APRs is much lower than the consumption reported for traditional approaches. One of the reasons is that the experiment we reproduce here [31] synthesizes ten candidate patches per bug-model, and applies perfect fault localization, which means that only the faulty statement is explored. For this reason, there is no energy "waste"on exploring areas of the research space not related to the bug.

Them, we observe that, as a difference from the traditional-APR based experiment, there is less variability on the energy consumption across the LLMs: The $Total_Energy$ metrics go from 564 Wh to 1,263 Wh, with an evident outlier (RepairLlama). From Figure 4 we also observe that there is a correlation between energy and repairability. For verifying this, we compute Spearman's correlation, giving as a result a correlation coefficient $r_s = 0.55$ (p-value=0.064). That value can be interpreted as *moderate correlation*.

We now focus on the models. We first observe that the largest models, in terms of amount of parameters, RepairLlama (which uses CodeLlama-7B),CodeGen6B and InCoder6B are the models that consume more energy and, at the same time, repair the largest number of bugs. The reason for consuming more energy includes the fact

that larger models execute more floating-point operations (as they have more parameters) which have higher hardware requirements (multiple GPUs). RepairLlama significantly improves the number of correct patches found a 65 w.r.t the other two models, from 37 patches to 61. This improvement is at expense of a 53% of energy increment from 825 to 1263 W.

We also observe that two models CodeT5Large and PLBARTLarge consume less energy (≈ 567 and ≈ 610 Wh, respectively) than the larger ones, e.g. InCoder1B and CodeGen2B, which consumed ≈750 Wh, and repaired more bugs than them. Using them could be a good compromise between energy consumed and repairability.

Table 2 presents the comparison of energy consumption between LLM-based approaches using Mann-Whitney test. We observe that there is significantly difference in energy consumption (those cells with p values < 0.05 -rejection of H0- colored black) between the three largest models (RepairLlama, CodeGen-6B, and InCoder-6B) and the other models. The effect size, measured using Cliff's Delta, is Small for most comparisons with a significant difference (52 cases), while the rest are Medium (20 cases).

Energy usage between the Inference and Validation phases from LLM-based APR tools. The LLM-based APRs considered in this experiment have two main phases: *1)* inference of patches (using the LLM model) and *2)* patch validation (here the model is not used any more). Figure 5 shows the sum of energy consumed in both phases (inference in blue, validation in orange). Most of the energy is consumed during the validation phase. Most of the models consume a similar amount of energy on that phase, with the exception of RepairLlama, which consumed more. A reason why a model consumes more energy in this phase compared to another is that it can produce more patches that compile (i.e., a model is better aligned with the developer expectations). As a result, these additional patches need to be validated, which increases the energy consumed.

In the inference phase, the largest models, RepairLlama, CodeGen-6B, InCoder-6B and CodeGen-2B consumes more energy than the others due to hardware requirements, since they need more than two GPUs. However, the energy consumed during the inference represents less than 20% of the total energy consumed. For other models such as PLBART or CodeT5, the impact of inference on energy consumption of the repair process is very low.

4.1.3 Consumption of a traditional APR tool compare to LLM-based APR tools. We compare TBar10C (introduced in Section 3.7.1) with LLM-based tools. Figure 4 shows that TBar10C consumes less energy than the three largest models evaluated in this experiment (RepairLlama using Codellama-7B, CodeGen-6B and InCoder-6B) but more than the others. TBar10C has a performance similar to CodeGen-2B, the third largest model considered in this experiment: TBar10C consumed a bit more energy (767 Wh vs 756 Wh), but repaired one more bug (31 vs 30). Also, as Table 1 shows in column $E/\#C$, TBar10C is more energy efficient per correct patch than CodeGen2B. In conclusion, this experiment shows that even being an approach from a paradigm that emerged before neural networks, TBar is still competitive and achieves results similar to those from new technologies such as the LLMs.

4.1.4 Answer to RQ1.

> Answer to RQ1: The state-of-the-art traditional approach TBar, based on a collection of existing templates, is the approach that most energy consumes to repair Defects4J bugs (118kWh). Furthermore, TBar reported the highest number of bugs (28). Beyond TBar, SimFix achieves a balanced trade-off between energy (72kWh) and rapairability (21 bugs) thanks to its customized search space.
> The three largest LLMs evaluated (CodeLlama7b -via RepairLlama, CodeGen6B and InCoder6B) are those that consume the most energy to repair Defects4J bugs. At the same time, they produced the largest number of correct patches. Consequently, attaining top performance on bug repair comes with increased energy consumption.

Just Accepted

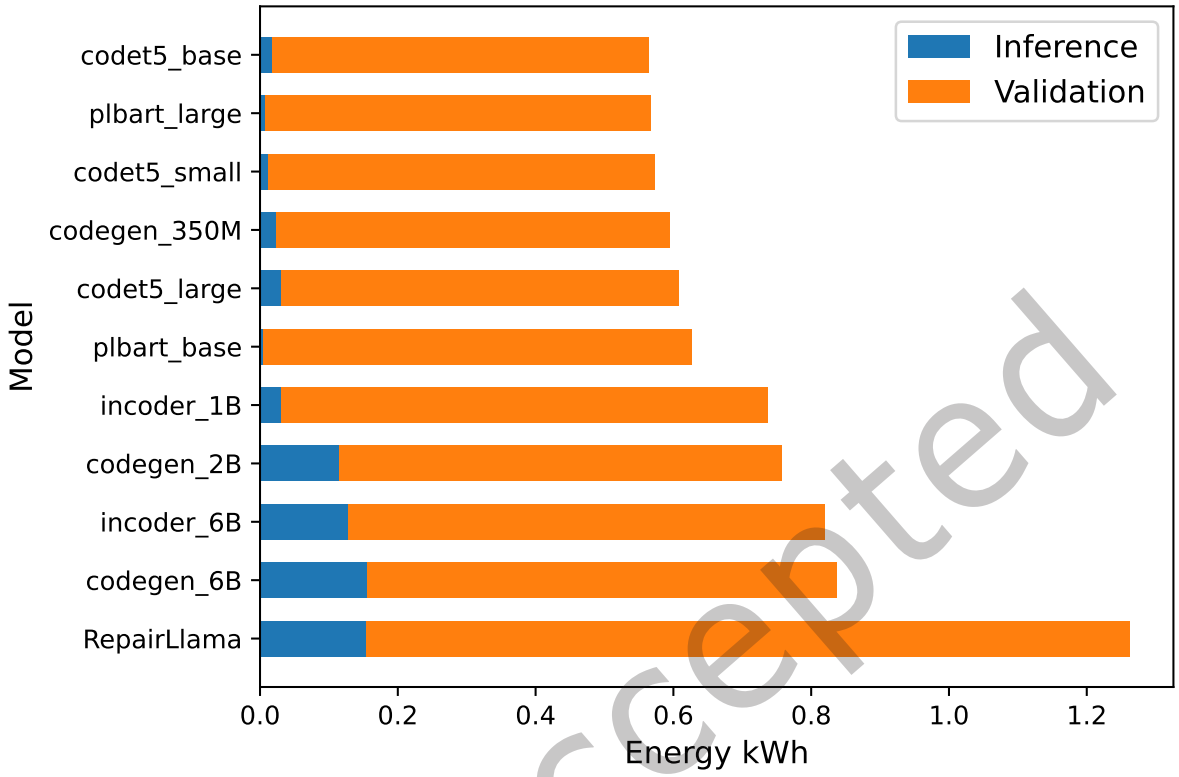


Fig. 5. Energy Consumption of LLM-based approaches.

Just Accepted

4.2 RQ 2: Which APR tool achieves the best energy efficiency in correctly fixing bugs

We focus on the trade-off between the total energy consumed divided by the number of first plausible patches classified as correct (column $E/\#C$ from Table 1).

From the traditional APRS, PraPR is the one that consumes less energy per correct patch. We recall that, as discussed in Section 4.1.1, it also minimizes the total energy consumed in the experiment. SimFix, TBar and FixMiner also exhibit a low $E/\#C$ value: Although TBar is the tool that most energy consumes, it is at the same time the most effective in terms of repair. These tools offer a good trade-off between the repairability and the energy consumed during the repair process.

This metrics gives low values for tools such as jMutRepair that repair a lower number of bugs, but the energy used is much lower than the used by others. A contrary case is for jGenProg, which is the second approach with the most amount of energy used, but its repair effectiveness is much lower than other approaches (just 5 patches correctly repaired with the first plausible patch found).

We now focus on LLM-based APRs. We group them in four groups according to their $E/\#C$. We introduce them in ascending order (from most efficient to least). First, PLBART-large, CodeT5-large, RepairLlama and CodeT5-base minimize the metric. As discussed in the previous section, they offer the best trade-off between energy and repairability. Second, the two largest LLMs, InCoder-6B and CodeGen-6B, consume more energy per correct patch than the three mentioned LLMs but are capable of correctly repairing more bugs. The rest of

Approach	Patches #Correct	$Energy_{Correct}$ Wh	$Energy_{Overfitting}$ Wh	$Energy_{Unrepaired}$ Wh	Total Wh
Avatar	17	1,818.9 (1.52%)	8,498.7 (7.12%)	107,235.3 (89.88%)	119,311.8
FixMiner	9	263.8 (0.60%)	5,667.4 (12.88%)	36,583.7 (83.11%)	44,017.4
Nopol	1	0.3 (0.00%)	11,334.4 (40.24%)	16,832.6 (59.76%)	28,167.5
PraPR	22	108.4 (3.03%)	1,117.2 (31.27%)	2,330.4 (65.22%)	3,573.2
SimFix	25	417.2 (0.50%)	5,722.1 (6.89%)	71,014.8 (85.47%)	83,083.2
TBar	33	4,761.5 (3.57%)	13,207.1 (9.90%)	111,218.7 (83.37%)	133,404.2
dynamoth	3	6.1 (0.01%)	13,159.5 (25.97%)	36,191.4 (71.42%)	50,676.0
jGenProg	5	22.2 (0.02%)	20,643.6 (15.54%)	109,635.0 (82.51%)	132,879.8
jMutRepair	4	6.9 (0.09%)	460.4 (6.12%)	7,046.5 (93.65%)	7,524.5
kPAR	9	327.1 (0.38%)	12,090.4 (13.97%)	71,743.6 (82.87%)	86,573.0
TBar10C	31	109.7 (14.31%)	52.5 (6.84%)	432.5 (56.40%)	766.9
RepairLlama	61	178.9 (14.17%)	114.2 (9.04%)	677.7 (53.67%)	1,262.7
CodeGen-2B	30	60.7 (8.02%)	80.6 (10.66%)	524.1 (69.30%)	756.2
CodeGen-350M	24	28.5 (4.79%)	29.1 (4.91%)	470.6 (79.24%)	593.9
CodeGen-6B	37	118.6 (14.16%)	63.6 (7.60%)	527.0 (62.94%)	837.2
CodeT5-base	28	49.9 (8.85%)	46.1 (8.17%)	384.4 (68.12%)	564.3
CodeT5-large	31	59.8 (9.84%)	69.2 (11.38%)	384.5 (63.25%)	607.9
CodeT5-small	18	27.2 (4.74%)	31.5 (5.51%)	474.9 (82.98%)	572.3
InCoder-1B	27	56.5 (7.67%)	96.6 (13.10%)	490.8 (66.60%)	737.0
InCoder-6B	37	96.3 (11.75%)	140.2 (17.11%)	456.2 (55.66%)	819.6
PLBART-base	24	27.8 (4.44%)	65.1 (10.39%)	453.3 (72.32%)	626.7
PLBART-large	30	71.9 (12.68%)	67.3 (11.87%)	323.7 (57.10%)	566.9

Table 3. Energy consumed by repair attempts that: a) repair correctly a bug (3th column), b) find overfitting patches (4th column), c) do not produce plausible patches (5th column).

Just Accepted

the LLMs except one are grouped in a third cluster. Lastly, CodeT5-small obtains the least energy efficiency per correct patch (31.7 Wh per correct batch). That is because its lowest repair ability (18 bugs) w.r.t. the others.

> Answer to RQ 2: PraPR and SimFix minimize the amount of energy per correct patch, without resigning repair effectiveness (308 and 3456 Wh per correct, resp.). Beyond being the approach with the highest energy consumed in the experiment, TBar consumes one of the lowest amounts of energy per correct patch (4234 Wh per correct).
> PLBART-large and CodeT5-large show a good trade-off between energy and repairability: these are the LLMs that consume less energy per correct patch found ($E/\#C$ equals 18.9 and 19.6 Wh, resp.) than some larger LLMs (e.g., InCoder-6B consumes 22.2 Wh). Even RepairLlama (CodeLlama7b) consumes more energy than the others with similar size (InCoder-6B, CodeGen-6B), it is more efficient ($E/\#C$ = 20.7) because it repairs a larger amount of bugs.

4.3 RQ 3: What is the impact of patch overfitting on energy consumption of APR?

Table 3 presents the results. Column `#Correct` shows the number of correct patches. Column $Energy_{Correct}$ shows the aggregation of the energy consumed by the repair attempts until the first **correct** patch is found. Column

$Energy_{Unrepaired}$ shows the aggregation of the energy consumed by repair attempts that just find **overfitting** patches. Column $Energy_{Unrepaired}$ shows the aggregation of the energy consumed on unsuccessful repair attempts, that is, those that do not find any **correct** patch. Column `Total` shows the total amount of energy consumed by a tool.[7]

The traditional APR send just a minor proportion of the total energy on attempts that correctly repair bugs. For example, the repair attempts from TBar which returned a correct patch used the 3.57% of the total energy consumed by TBar in this experiment. From ≈6% to ≈40% of energy is consumed by repair attempts that only find overfitting patches. The approaches that consume more energy on such attempts are those that generate the largest number of plausible patches (as shown in Table 1), such as Nopol, which produced 91 plausible patches for bugs but correctly repaired only one. Finally, most of the energy consumed by a repair tool (between ≈60% and ≈93%) corresponds to repair attempts that did not find any patch (neither correct nor overfitting).

For the largest LLMs, RepairLlama (CodeLlama 7B), InCoder-6B and CodeGen-6B, between 11.75% and 14.17% of the total energy consumed during the experiment was consumed to find the first correct patch. This percentage is higher than those of traditional tools because the LLM-based tools use perfect fault localization and thus energy is not "wasted" in proposing candidate patches for non-buggy locations.[8] The energy from repair attempts that produce only overfitting patches range from ≈5% to 17%. Finally, similar to what was observed with Traditional APRs, most of the energy (between ≈53% and 79%) is consumed on repair attempts that did not find any plausible patch.

Overfitting patches not only harm the accuracy of a repair APR [59, 60] but "waste" energy from a repair attempt. We also calculate the "wasted" energy consumed once an overfitting patch is synthesized, specifically, during the validation phase. We focus on overfitting patches found before a correct one (if any), or in those from repair attempts that fail to correctly fix a bug. The results are shown in Table 4. We observe that the validation phase of overfitting patches consumes less than 1.28% of the total energy for traditional APRs, and between 1.48 and 4.43% on LLM-based APRs. This means that most of the energy is consumed on other phases such as patch search, code synthesis/generation and patch compilation. Utilizing overfitting detection techniques, such as ODS [69], to discard patches prior to the patch validation phase could result in modest energy savings.

> Answer to RQ 3: Repair attempts that successfully find a correct patch consumed a small proportion of the total energy used in the experiment: less than 3.57% for traditional methods and less than 14.2% for LLMs. The remaining energy is consumed by unsuccessful repair attempts, either through the generation of overfitting patches or the failure to produce any plausible ones, leading to wasted energy. Validating overfitting patches consumes at most 4.5% of the energy. Therefore, discarding these patches with classification techniques before validation (e.g. [69]) may not lead to significant energy savings.

4.4 RQ 4: To what extent does the energy consumed to find a patch correlate with: a) the repair time?, b) difficulty to fix a bug, c) the number of patch candidates (NPC) and d) project under repair?

4.4.1 Repair Time. We calculate the Spearman's correlation between time and energy to find the first plausible patch. Column T_{fp}-E_{fp} from Table 5 shows the respective values. For each APR tool, whether traditional or LLM-based, the Spearman correlation coefficient (R) returned a value between 0.93 and 1, which means that consumption is fully positive aligned with execution.

[7]Total includes the energy consumed after finding the correct patch until the stopping condition (e.g. timeout) is reached.

[8]Note that $Energy_{Correct}$ includes the energy consumed until the first correct patch. The energy consumed to validate other correct patches after the first is included in the Total column but not in $Energy_{Correct}$.

Just Accepted

Approach	Patches #Overfitting	$Energy_{overfitting}$ Wh	Percentage total
Avatar	535	213.8	0.18%
FixMiner	115	54.7	0.12%
Nopol	197	187.6	0.67%
PraPR	106	38.9	1.09%
SimFix	98	99.8	0.12%
TBar	513	342.6	0.26%
dynamoth	154	102.7	0.20%
jGenProg	3906	1,704.2	1.28%
jMutRepair	88	50.2	0.67%
kPAR	469	240.9	0.28%
TBar10C	19	15.0	1.96%
RepairLlama	52	40.7	3.22%
CodeGen-2B	19	29.5	3.90%
CodeGen-350M	7	8.8	1.48%
CodeGen-6B	22	26.0	3.10%
CodeT5-base	20	23.2	4.11%
CodeT5-large	19	21.3	3.50%
CodeT5-small	8	12.1	2.11%
InCoder-1B	28	32.7	4.43%
InCoder-6B	28	29.6	3.61%
PLBART-base	16	17.9	2.85%
PLBART-large	14	17.0	3.00%

Table 4. Energy consumed for validating overfitting patches. The column `Percentage total` represents the proportion of that energy over the total energy consumed.

4.4.2 Energy and NCP (number of patch candidates). Column NPC_{fp}-E_{fp} from Table 5 shows the correlation between the energy required to find the first plausible patch and the number of patch candidates (NPC) before the first plausible patch is found. Let us first focus on traditional APRs. If we consider all tools together (Row All-Traditional), the correlation is 0.56, which means a moderate correlation between NPC and energy. However, considering each APR separately, the correlation has divergence values. For some tools, such as Avatar or TBar the correlation is high (>0.75). For other tools such the correlation is moderate (e.g.,as SimFix) or low (jMutRepair)

For LLM-based, the correlation between energy and NPC considering all LLMs is also moderate (0.59). We observe that most LLMs have a moderate correlation, which goes from 0.39 to 0.68. The exception to this is CodeT5Small, with a slightly higher correlation (0.77).

In conclusion, the correlation between NPC and energy is, in general, moderate for both traditional and LLM-based tools families. Consequently, NPC is not an accurate proxy of the energy consumption of program repair.

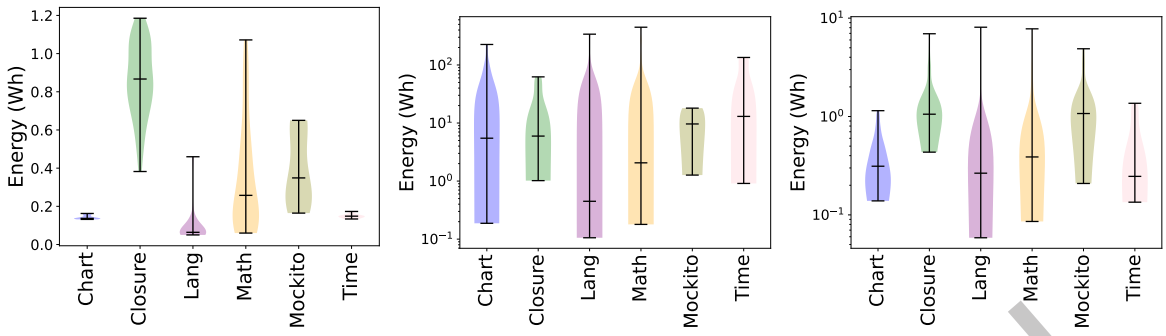
4.4.3 Energy and Difficulty of Repair (time-to-fix). The median time-to-fix, in days, of the bugs with correct patches is half (0.5) a day for both traditional and LLM based-tools. Column T2F-E_{fcp} from Table 5 shows the correlation coefficients. For LLM-based tools, there are very weak (e.g. PLBARTBase) or weak positive correlation (e.g., RepairLlama) between energy and time-to-fix, with some exceptions with positive moderate correlation (e.g. CodeT5Base). For most of the traditional APRs, the correlation is very weak (e.g. PraPR) or weak (e.g., kPAR), but also there are some outliers such as a moderate positive correlation (jGenProg), a moderate negative correlation (jMutRepair) and a very strong correlation (Dynamoth).

Just Accepted

	E_{fp}	T_{fp}	T_{fp}-E_{fp}	NPC_{fp}	NPC_{fp}-E_{fp}	T2F	T2F-E_{fc}	Pr	Pr-E_{fc}
	md	md	crl	md	crl	md	crl	md	crl
Nopol	4079	71.06	0.98 (91)	14.00	0.64 (91)	fncp	fncp	fncp	fncp
PraPR	3991	76.82	1.00 (75)	error	error	1.29	0.12 (16)	3.00	0.32 (14)
FixMiner	14588	340.24	0.99 (26)	28.00	0.60 (26)	0.12	0.07 (7)	2.00	-0.58 (7)
kPAR	30933	692.17	0.99 (40)	59.50	0.67 (40)	1.72	0.36 (8)	3.00	-0.58 (7)
SimFix	12570	215.06	1.00 (52)	12.50	0.38 (52)	0.17	0.36 (18)	3.00	0.08 (17)
jMutRepair	3927	68.78	0.93 (32)	17.50	0.12 (32)	0.50	-0.50 (3)	3.00	0.87 (3)
dynamoth	4577	82.39	0.99 (61)	5.00	0.48 (61)	23.45	1.00 (2)	3.00	nan (2)
TBar	44880	1065.46	0.99 (71)	101.00	0.83 (71)	0.50	-0.08 (23)	3.00	-0.12 (22)
Avatar	32070	641.69	0.99 (45)	131.00	0.76 (45)	0.89	0.27 (10)	3.00	-0.37 (9)
jGenProg	6675	123.05	0.99 (40)	17.00	0.00 (40)	0.41	0.50 (3)	3.00	0.87 (3)
All traditional	6559	128.6	0.99 (533)	23.5	0.56 (458)	0.5	0.11 (91)	3.0	-0.13 (85)
TBar10C	2277	39.54	0.98 (39)	3.00	0.39 (39)	0.50	-0.12 (19)	3.00	-0.00 (19)
RepairLlama	2566	32.54	0.95 (77)	1.00	0.57 (77)	0.55	0.28 (36)	3.00	-0.06 (36)
codegen350M	1512	32.84	0.95 (27)	1.00	0.64 (27)	0.34	0.23 (16)	3.00	-0.11 (16)
codegen2B	2192	37.91	0.95 (39)	2.00	0.50 (39)	0.50	0.35 (17)	3.00	-0.41 (17)
codegen6B	3247	41.62	0.94 (45)	2.00	0.56 (45)	0.33	0.20 (19)	3.00	-0.13 (19)
codet5small	2234	45.47	0.95 (23)	2.00	0.77 (23)	0.34	0.13 (15)	2.00	-0.28 (15)
codet5base	1680	32.80	0.96 (34)	2.50	0.68 (34)	0.59	0.50 (19)	3.00	0.12 (19)
codet5large	2189	39.66	0.94 (42)	2.50	0.61 (42)	0.34	0.40 (19)	3.00	0.08 (19)
InCoder1B	2253	38.99	0.98 (44)	2.00	0.59 (44)	0.41	0.07 (16)	3.00	0.15 (16)
InCoder6B	3269	47.47	0.94 (57)	2.00	0.62 (57)	0.23	0.29 (24)	3.00	0.26 (24)
plbartbase	2433	42.92	0.97 (34)	2.00	0.75 (34)	0.31	0.12 (16)	2.00	-0.53 (16)
plbartlarge	3006	59.74	0.98 (41)	3.00	0.68 (41)	0.59	0.23 (21)	3.00	0.10 (21)
All LLM-based	2408	39.5	0.95 (502)	2.0	0.59 (502)	0.5	0.15 (237)	3.0	0.02 (237)

Table 5. RQ 3: Spearman correlation between variables Energy E_fp and Repair Time T_fp, Number of Candidate Patches NCP_fp, Time-To-Fix (by human) and Priority Pr. The parentheses show the total number of bugs considered in the correlation. Columns *md* show median values. The columns *crl* show the correlation coefficients. The value $fncp$ means that all patches found are incorrect, thus making it impossible to compute correlation. An *error* means that we could not extract the data for the tool to compute the correlation.

Just Accepted



(a) Distribution of energy require to run all test from a Defects4J bug.

(b) Energy correct patch from traditional tools (log scale).

(c) Energy correct patch from LLM-based tools (log scale).

Fig. 6. Distribution of energy across projects.

4.4.4 Energy and Difficulty of Repair (priority). Column Pr-E$_{fcp}$ from Table 5 show the correlation coeficcients between the energy consumed for correctly repaired bugs and the priority assigned to these bugs' issue in the bug tracking system. The priority is represented with a scale of 1 (lowest) to 5 (higher) [26].

We observe a disparity on the coefficients: some APR tools have a negative moderate correlation, that is, more energy required, less priority (e.g., PLBARTBase, FixMiner), while other tools have moderate positive correlation, that is, the more energy required, the higher priority (e.g., PLBARTLarge, InCoder2B, CodeGen2B). Surprisingly, we observe some discrepancy between the coefficients of LLMs of the same family (e.g., PLBARTbase vs. PLBARTlarge, CodeT5small vs. CodeT5base, CodeGen2B vs. CodeGen6B).

4.4.5 Energy and Projects under repair. First, we study the energy consumed by test suites from Defects4J. We recall that test suites are used to validate candidate patches.

Figure 6a shows the energy distribution to execute the entire test suite of a bug from Defects4J. We observe that the energy required and the distribution vary across the Defect4J projects. In particular, validating a candidate patch on Closure, Math and Mockito projects (mean 0.81Wh, 0.35Wh and 0.34 resp.) requires, in general, more energy than on Lang, Chart and Time (0.08, 0.14 and 0.15 resp.). In addition, the distributions are different: in projects such as Mokito or Closure, the energy consumption of tests significantly varies across bugs from these project (e.g., σ = 0.21Wh for Closure), while in Chart and Time the test suite executions consume a similar amount of energy across their bugs (σ = 0.01Wh).

As a follow on, since we have observed that the energy consumption of running test suites varies across bugs, we investigate whether this variation affects the energy consumed to repair a bug.

Figures 6b and 6c show the distribution (log scale) of the energy required by traditional and LLM-based tools, respectively, to find the first correct patch, grouped by projects. In both families, we observe that: *a)* There are groups of projects that have similar distribution shape. For instance, Chart, Lang and Math in the case of traditional approaches, compared to Closure and Time; the similarity of Lang and Math also can be observed in LLM-based tools, although in this case not with Chart *b)* Medians also define groups of projects: Chart, Closure, Mockito and Time have higher median than the others in the case of traditional approaches, while in LLM-based it is just Closure and Mockito that keep this higher median.

The variation of energy required to fix a bug is due to several reasons, some of them are: *a)* As discussed at the beginning of this section, test cases from bugs consume different amount of energy. This means that, for instance, a tool that generates three patches for a bug can eventually consume much more energy to find the same amount

Just Accepted

of patches -three- for another bug. This also explains, as seen in Section 4.4.2, that the metric *NPC* is not an accurate proxy of energy consumption. *b)* The search space of each tool is different: Some have optimized spaces, such as SimFix, helping to reduce the energy required to find the correct bug, as shown in Section 4.1.1.

Answer to RQ4:
From our analysis of Defects4J bugs we observe that there is a very high correlation between repair time and the energy consumed by both traditional and LLM-based tools, but moderate correlation between NPC and energy, and weak correlation between energy and bug priority, and also weak between energy and time-to-fix. The energy footprint of the bugs depends not only on the project to which it belongs but also on the buggy versions under repair.

5 DISCUSSION

5.1 Implications

Implications for researchers. The evolution of the field APR has been driven by effectiveness metrics such as precision metric, which reports the number of fixed patches over the total. Other (few) metrics have also been reported, such as repair time or number of patch candidates. In this work, we show that these metrics are not enough to outline the sustainability of APR tools. Although we agree that these metrics are crucial to achieve *better* APR tools (i.e., the precision increases with the release of state-of-the-art APRs), we consider that improvement cannot ignore the consequence that it produces in our environment. We encourage the inclusion of new metrics in future APR experiments in order to illustrate the energy profile of future state-of-the-art approaches.

The difference in energy consumption of APR that this study showed suggests some challenges for researchers: *a)* Measure and report the energy consumed by an approach: this is not a straightforward activity because their authors/developers have to modify their experimental platform in order to measure the energy; *b)* optimize the search space or the model: there is a trade-off between repairability and energy required. The challenge is to define a "boundary" between them. For example, on a template-based APR, one template can help to repair one particular bug in Defects4J, increasing its precision. However, its usage on other bugs (which the template does not help to repair) produces a waste of energy. This challenge focuses on that: to determine, beyond the repairability, the impact that a template has in our environment and to determine to what extent including it is sustainable or not. *c)* Integrate overfitting tools for patch analysis (especially done that carry out static analysis such as [67, 69]), and execute them before patch validation using the test suite, in order to save energy from evaluations of overfitting patches. *d)* Optimizes the execution of the test suite: Even though there exist techniques of test prioritization in the literature [100], researchers rarely apply them in the implementation of their repair tools.

Implications for practitioners. Energy has an operational cost of APR that the practitioner needs to afford (an indirect cost if the deployment is in the cloud and direct if it is on-premise). Our study offers practitioners a first study of the underlying energy cost of APR tools in Java. We consider the energy profile of the APR tools to be relevant for practitioners. First, the energy consumed by a tool is not negligible: we show that, for instance, the state-of-the-art TBar tool consumes, on average, the energy consumed by a smartphone during 2 days of heavy usage. The energy cost of APR can increase for practitioners if we consider the integration of APR tools in production: APRs connected to CI/CDs (e.g., via a bot [21, 101]). This setup invokes the APRs tools for each failing build caused by test cases with failure or error. For example, the study by Moritz et al. [22] in Travis CI builds reports a mean of 10.3% for builds with failed tests in Java projects. With that setup, each failed build triggers one or more APR tools. The deployment of a greedy APR tool can have a negative impact on the energy consumed by

Just Accepted

the CI/CD. To obtain a sustainable development process, practitioners should be aware of the energy profile of the tools they deploy. The findings presented in this paper take a first step in that direction.

5.2 Energy Consumption of Inference on GPU vs CPU

In our experiment, we use NVIDIA V100 GPUs for LLM inference. We also explore inference using only the CPU and compare the energy consumption of the two approaches. We chose RepairLlama for this experiment because its scripts, unlike those provided by [31], allow for CPU-based inference with minimal modifications.

The energy consumed during inference on the CPU is 6.1 times higher, based on the median, than that consumed by the GPU (GPU: median 4.03 Wh, mean 6.472 Wh; CPU: median 24.66 Wh, mean 30.2375 Wh). We also conducted a Wilcoxon paired test to evaluate the null hypothesis: *"There are no significant differences between the energy consumed by the CPU and GPU during the inference task."* At a significance level of 0.05, we reject the null hypothesis.

In conclusion, inference using the CPU is feasible for the patch synthesis task but comes at the expense of a significant increase in energy consumption. To enable Green APR solely on the CPU, it would be necessary to apply techniques such as model pruning, quantization, and compression, and to evaluate the potential loss of accuracy.

5.3 Impact on Data Memorization on Green APR

Data leakage refers to the use of information during model training that would not normally be available during prediction, resulting in inflated performance metrics that misrepresent the model's true effectiveness [102]. A code model is considered to memorize a string from its training data if there exists a prompt that causes the model to generate this string [103].

In the context of program repair, memorization occurs when a generated patch for a buggy program (e.g., a bug from the Defects4J benchmark) matches the canonical solution available in the benchmark. Recently, Ramos et al. [102] studied the extent to which LLMs memorize bug benchmarks such as Defects4J. Their study included two LLMs analyzed in our paper: CodeGen6B and CodeLlama7B. Their findings show that both models exhibit significant evidence of memorization in Defects4J.

From the APR repairability perspective, memorization has a negative impact because it inflates certain metrics, such as accuracy: an LLM has memorized testing data, including correct fixes of bugs used in the APR evaluation. From the Green APR perspective, memorization also has a negative impact because it makes the APR process less 'green'. When a model memorizes a solution for a bug during training (e.g., CodeGen6B memorizing Lang-39 from Defects4J, as shown by [102]), an experiment aimed at repairing that bug with the model —like ours— would likely produce the correct patch. However, a model that did not memorize the bug would likely generate initial patches that differ from the canonical (correct) patch. If those patches are incorrect, the APR would spend energy on compilation and validation, unlike in the memorization scenario, where no energy for these steps is required.

The adoption of techniques to avoid memorization (such as those from [103]) would not only provide a better estimate of APR accuracy but also offer a more precise assessment of its sustainability in practice.

6 GUIDELINES FOR GREENER AUTOMATED PROGRAM REPAIR

A key challenge is how to make greener APR. Other communities, such as the AI community, have already focused on making greener assets, such as LLMs [104]. In the context of APR, Zhang et al. [39] emphasize the need to pay more attention to the repair costs of LLMs and propose three key challenges to address this issue: *1)* optimizing the training of LLMs, *2)* optimizing patch generation (i.e., the inference), and *3)* optimizing patch validation.

Just Accepted

In this section, we align the three identified challenges from Zhang et al. [39] with Green APR, and present guidelines consisting of a series of recommendations for developing more energy-efficient APR.

6.1 Recommendation: Apply Test Sampling and Refactoring

As Figure 6a shows, the energy consumed during test cases execution depends on the project under repair. This means that the amount, length, or quality of the test can have an impact on the energy consumed to run the tests. The third challenge written by Zhang et al. focuses on patch validation. As demonstrated in this experiment (illustrated in Fig. 5), running the entire test suite during patch validation may waste energy, as some functionality might remain unaffected by the bug being repaired.

Therefore, our recommendation for reducing energy on APR is to apply techniques for reducing the number of test cases executed during the repair process. For example, an APR tool could selectively run only the test cases that exercise the suspicious buggy line, method, or class.

This reduction strategy can be further enhanced with test refactoring techniques, particularly those designed for APR. For example, B-Refactoring [105] is a technique for automatically refactoring test code to improve the dynamic analysis performed by the repair approach Nopol [47]. By applying this technique, an APR can rely on smaller, more focused test cases, executing only those relevant to the bug, thereby reducing energy consumption.

6.2 Recommendation: Apply Overfitting Detection Techniques

As we observe in Fig. 5, most of the energy is spent on patch validation. This is aligned with the third challenge written by Zhang et al. [39], which focuses on patch validation. To address this challenge, they recommend enhancing patch validation with advanced strategies, such as mutation testing, or leveraging LLMs to rank candidate patches before validation.

Our recommendation for reducing energy consumption is to incorporate overfitting detection techniques as an additional step in the repair workflow. For example, a binary classifier could be applied immediately after patch synthesis and before test case validation, which is computationally expensive. If the classifier confidently identifies a patch as overfitting, the repair process can skip test execution, saving energy. ML-based approaches, such as ODS [69] and CACHE [71], have demonstrated promising results in patch classification.

Additionally, using static techniques like ODS -which does not require execution- for avoiding overfitting patches could further reduce energy on the preparation of the patched version before the validation, which include, for example in Java repair, the compilation of the patched version.

6.3 Recommendation: Apply Test Prioritization

Executing the entire test suite during APR is energy-intensive, as many test cases may be unnecessary for validating a patch. We recommend test prioritization to address this issue by running the most relevant test cases first, reducing computational overhead and energy consumption. By focusing on tests more likely to assess patch correctness, the repair process can become more efficient while maintaining reliability. This recommendation addresses the third challenge by Zhang et al. [39]. To our knowledge, all the APR tools we consider in our experiment do not apply test prioritization. In the context of Defect4J benchmarking, most of these tools execute the command `defects4j test` which does not implement test execution prioritization[9]. Other tools apply simple prioritization schema, such as PraPR which sorts the test cases based on their previous running time in ascending order [24]. Implementing more sophisticated techniques is crucial for achieving greener APR.

[9]https://defects4j.org/html_doc/d4j/d4j-test.html

Just Accepted

6.4 Recommendation: Apply Ranking on Candidate Patches

Previous work, such as Prophet by Long et al. [45] for C language repair, has applied ML approaches to rank patches and present the most likely correct ones to developers before the incorrect ones.

We recommend incorporating patch ranking within the repair process, for example, after a set of patches is generated for a suspicious line. Once ranked, the repair approach could evaluate the *top-k* patches, avoiding the energy expenditure of validating patches that, according to the ranking, are likely to be incorrect. This recommendation also addresses the third challenge by Zhang et al. [39].

To our knowledge, among the traditional tools we considered, only PraPR [24] applied a ranking based on the suspiciousness value of the locations where patches are applied. We believe there is an opportunity to implement ML-based ranking approaches, similar to the preliminary work by Long et al., to optimize the energy consumption of APR tools.

We also recommend to measure the impact of introducing these new techniques into the repair workflow. Zang et al. suggest to address their third challenge (the patch validation) by enhancing patch validation with advanced strategies, such leveraging LLMs to rank candidate patches before validation. However, from a green perspective, adding extra calls to LLMs could eventually have negative impact. To measure the benefits of the enhancing patch validation, it would be necessary to measure the energy consumed (E_{llm_val}) for this additional task (e.g. call to LLMs as suggested by Zang et al.) and compare it to the energy saved by skipping the validation of the incorrect patches (E_{saving_val}). Consequently, an APR equipped with such a patch validation extension would be considered greener than the original APR if $E_{llm_val} < E_{saving_val}$.

6.5 Recommendation: Apply Repair Actions Prioritization

The search space of APR tools can be vast [106], and for this reason, previous research has proposed techniques to facilitate navigation within these spaces in order to accelerate the search for the correct patch [42, 44].

We recommend applying techniques to prioritize repair actions (or 'mutations') to avoid applying operations that do not lead to correct patches, thus saving energy by reducing the validation of incorrect patches. To our knowledge, from the traditional tools we consider; only PraPR [24] prioritizes the mutations used to generate candidate patches, with prioritization based on the frequency of these mutations in patch datasets. This recommendation addresses the second challenge by Zhang et al. [39] (patch generation).

6.6 Recommendation: Conduct Efficient Training and Tuning

The LLM-based repair approaches considered in this research have been re-trained (fine-tuned) with bug fixes for the downstream task of program repair. To address the first mentioned challenge (training optimization), Zhang et al. [39] propose applying parameter-efficient fine-tuning approaches to APR. To the best of our knowledge, Silva et al. are the first researchers to apply this technique in the APR field, introducing RepairLlaMA, based on CodeLlama-7B, a tool included in our experiment. In particular, RepairLlaMA employs a parameter-efficient fine-tuning technique called LoRA [107] to train repair adapters.

We recommend applying efficient fine-tuning, as done by Silva et al., rather than full fine-tuning as performed by Jiang et al. [31]. This approach would be more energy-efficient while simultaneously improving repairability.

6.7 Recommendation: Apply Inference Optimization

To address the second challenge from Zhang et al. [39] (inference optimization), they suggest optimizing the size of LLMs without significantly compromising their performance by applying techniques such as model pruning, quantization, and knowledge distillation. Previous work has empirically studied the impact of some of these techniques on code generation tasks. For example, Wei et al. [108] studied the impact of quantization on LLMs with up to 7 billion parameters. However, to the best of our knowledge, and as corroborated by the literature

Just Accepted

review by Zhang et al. [39], none of these techniques have been applied to LLM-based APR. Beyond quantization and pruning techniques, recent work has introduced novel compression methods for code models. For example, Compressor [109] and Avatar [110] have successfully optimized models such as CodeBERT and GraphCodeBERT. In particular, Avatar optimizes these models in terms of model size (160x smaller, achieving a size of 3MB), inference latency (up to 76x faster), energy consumption (up to 184x less), and carbon footprint (up to 157x less), with a negligible loss in effectiveness (1.67% on average).

Our recommendation is to create or reuse specific benchmarks to evaluate performance, along other metrics such as energy consumption of the models used by the APR. For example, Silva and Monperrus [111] have recently proposed an infrastructure to benchmark the repairability of LLMs.

Based on these benchmarks, we also recommend focusing on the frameworks and libraries used for inference. For example, a recent study by Duran et al. [112] measured the impact of LLM runtime engines and execution providers on energy consumption during the source generation task. Thus, making a strategic decision about the execution provider used by the repair tool for patch inference could lead to energy savings.

Moreover, we recommend applying hyperparameter optimization to the inference process. For example, as reported by Martinez [113], optimizing parameters such as batch size, number of GPUs, or the inference engine can impact inference performance (measured in that paper as throughput). Similarly, this parameter optimization can be guided by a focus on minimizing energy consumption.

6.8 Recommendation: Apply LLM Prompt Engineering

Recent work by Hidvégi et al. [114] introduced CigaR, the first LLM-based APR tool designed to minimize the computational cost of repairs, measured by the number of tokens used. Reducing token usage can also lead to energy savings, as the underlying LLM requires less computational power to process a more efficient tokenized input.

We recommend that LLM-based APR tools adopt the techniques introduced in CigaR (and their future evolutions), which focus on crafting concise yet informative prompts to minimize token cost. In CigaR, this is achieved through three key prompts: the *initiation prompt* the *improvement prompt* and the *multiplication prompt*.

We also recommend exploring and integrating various prompt engineering techniques into APR tools. For example, Rubei et al. [115] demonstrated that prompt engineering on the LLaMA model can reduce LLM energy consumption during inference without compromising performance. This reduction was achieved by augmenting prompts with dedicated custom tags and explanations, effectively lowering energy usage. These recommendation address the second challenge by Zhang et al. [39] (patch generation).

6.9 Recommendation: Apply repair actions at lower-level or mega-program

We observe in this paper that PraPR [24] is the approach that less energy consumes (Fig 3). One design decision that produce that is that it works at the level of code: it creates candidate patches by applying *mutations* directly into the bytecode. This means that the patched code does not need to be compiled, and it can be direcly validated with test cases. This stategy helps to save time and energy from the compilation process. The other traditional APRs apply the repair actions in the buggy source code file. This means that these approaches need to compile the patched file each time it applies a candidate patch. In programs with a vast search space, the amount of energy spent doing this process could be significant.

We also recommend applying bundling of candidate patches into a meta-program containing multiple candidate patches that is compiled only once, with the runtime selecting which patch to activate (one at a time). Meta-programming has been explored in program repair by only a few works, including JAID [116]. These recommendation address the second challenge by Zhang et al. [39] (patch generation).

Just Accepted

7 THREATS TO VALIDITY

Dataset of bugs. We focus on the first version of Defects4J [11] as it is one of the most used bug data sets used by the community [37]. Recent work in APR has also used, in addition to the previously mentioned, other datasets (including the newest version of Defects4J). However, adding new bugs or a dataset rapidly increases the magnitude of the experiment, and this is a major impact on the limited computational cost and time we have. For this reason, we decided to focus on the most used dataset, DefectsJ4 v1.0, and let further experiments on additional datasets (including bugs from other programming languages) be made for future work.

Selected APR tools. We focus on the energy required to fix Java bugs. Although there have been many previous works that focus on repairing Java (as shown by [37]) not all tools are available or work properly [87]. For this reason, we decided to base our experiment on two previous studies. Liu et al. [19] reproduce traditional Java APRs tools presented in other papers. This work provides a distilled version of the tools that can be used. It covers different techniques, from constraint-based tools such as Nopol [47] to template-based ones such as TBar [27]. We started our study from that list. The second study by Jiang et al. [31] provides scripts that use LLMs such as Incoder or CodeGen. We acknowledge that there exist other approaches that could be included in this paper, such as neural network-based approaches, such as [20, 52, 94]. Their performances are similar to those presented by the LLM-based APRs, for that reason, as we have limited computational resources, we chose the latter.

Number of repetitions. We applied five repetitions by repair attempt, which is less than a recommendation [98]. The large magnitude of our experiment, given the number of tools, bugs, and execution time, and our limited computational budget in the cluster, defined that number of repetitions. As a mitigation action, we have applied a strategy to remove outliers: we check that power consumption at the beginning or an attempt does not differ from the power that a used cluster's node consumes on idle mode.

Results from the repair attempts. In our experiment, some repair attempts failed to find a plausible patch which were reported in previous works, for example [19]. The main reason for missing plausible patches could be the execution time: five hours may not be enough to find the patch. Note that the previous replication study [19] does not use a time budget for halting the executions, but a prefixed number of operations, which could take longer than five hours.

Manual evaluation. We analyze the patch labeled we reused from previous work [17, 31], finding few disagreements (as reported by [67]). For that reason, the number of correct patches could vary w.r.t the literature.

8 LIMITATIONS

This paper has some limitations. The first one is the tools used in our experiment. In case of traditional APR, we focus on Java written tools for repairing Java bugs. There exist other tools written in other languages and/or that focus on repairing programs written in other languages. For example, FixKit [117] is written in Python for Python bugs. For LLM-based approaches, there are other neural-network based approaches (such as ITER [118]) or other open-source LLMs (StarCoder, Mistral) that could be used. Our study is limited to tools that use open-source LLMs (deployed locally) and excludes tools, such as ChatRepair [119], that use closed-models (e.g. OpenAI models) accessed via API. The main reason for this exclusion is that we cannot compute the energy consumed by third-party actors during inference. The number of APR tools included in our large-scale experiment is strongly limited the amount of computation required to reproduce the experiment. This limitation on resources also affects on the amount of bugs that we consider. We are also limited by the hardware available specially by the GPU devices used for deploying LLMs. In this experiment, we used a Nvidia V100[10], which is the first Tensor Core

[10]Nvidia V100 whitepaper: https://images.nvidia.com/content/volta-architecture/pdf/volta-architecture-whitepaper.pdf

Just Accepted

GPU introduced by Nvidia designed specifically for deep learning. Upgrading the GPU model to A100 or H100 could eventually reduce energy consumption at expenses of increasing the infrastructure costs.

9 CONCLUSIONS AND FUTURE WORK

In this paper, we combined two research fields: APR and Green software development. We measure the energy consumed to find correct patches from 10 traditional tools and from 11 fine-tuned LLMs models. Our analysis revealed a moderate positive correlation between energy consumption and repairability, suggesting that APR tools require more energy as their success in repairability increases. Furthermore, we demonstrate that patch overfitting —one of the major challenges in the APR community— not only undermines the reliability of APR tools but also increases energy consumption during the repair process.

In future work, we plan to focus on, for instance:

- to optimize existing APR tools from the point of view of energy consumption;
- to develop new hybrid repair methods that integrate traditional approaches with LLMs, focusing on reducing the energy consumption of the repair process. This includes, for example, leveraging LLMs to generate repair templates -such as those used by TBar [27] and Cardumen [92]- as well as enhancing template-based APR with LLM assistance in patch generation and validation;
- to understand the accessible minimum consumption for the given category of APR tools;
- to propose design decisions of APR tools in order to make APR more energy efficient;
- to study the relation of type of bugs and energy required to repair them;
- target bugs written in other languages (e.g., Python, C) and benchmarks;
- to carry out a fine grained monitoring of the energy consumed by a APR tool in order to detect the consumption by, for instance, stages (fault localization, synthesis, validation) or components;
- to investigate the impact of LLM memorization on the energy efficiency of APR;
- to replicate experiment in other hardware and to find trade-off between tool requirement, energy consumed and infrastructure cost;
- to define energy labels for APR tools and label existing tools;
- to define new metrics of energy consumption specially target for program repair;
- to focus on the sustainability of repair strategy -which are implemented by the APR tools-.

ACKNOWLEDGES

This paper has been funded by the “Ramon y Cajal” Fellowship (RYC2021-031523-I) and the GAISSA Spanish research project (ref. TED2021-130923B-I00; MCIN/AEI/10.13039/501100011033).

REFERENCES

[1] C. Calero, M. Moraga, and M. Piattini, “Introduction to software sustainability,” in *Software Sustainability*. Springer, 2021, pp. 1–15.
[2] C. C. Venters, R. Capilla, S. Betz, B. Penzenstadler, T. Crick, S. Crouch, E. Y. Nakagawa, C. Becker, and C. Carrillo, “Software sustainability: Research and practice from a software architecture viewpoint,” *Journal of Systems and Software*, vol. 138, pp. 174–188, 2018.
[3] C. Becker, R. Chitchyan, L. Duboc, S. Easterbrook, B. Penzenstadler, N. Seyff, and C. C. Venters, “Sustainability design and software: The karlskrona manifesto,” in *2015 IEEE/ACM 37th IEEE International Conference on Software Engineering*, vol. 2. IEEE, 2015, pp. 467–476.
[4] C. Calero and M. Piattini, *Green in software engineering*. Springer, 2015, vol. 3.
[5] P. Grünbacher and Y. Ledru, *ERCIM News – Special Theme: Automated Software Engineering*. ERCIM EEIG, B.P. 93, F-06902 Sophia-Antipolis Cedex, France, 2004.
[6] D. H. O’Dell, “The debugging mindset: Understanding the psychology of learning strategies leads to effective problem-solving skills.” *Queue*, vol. 15, no. 1, p. 71–90, feb 2017. [Online]. Available: https://doi.org/10.1145/3055301.3068754
[7] H. Krasner, “The cost of poor software quality in the us: a 2020 report,” *Proc. Consortium Inf. Softw. QualityTM (CISQTM)*, 2021.
[8] C. Le Goues, T. Nguyen, S. Forrest, and W. Weimer, “Genprog: A generic method for automatic software repair,” *Ieee transactions on software engineering*, vol. 38, no. 1, pp. 54–72, 2011.

Just Accepted

[9] F. Long and M. Rinard, "Staged program repair with condition synthesis," in *Proceedings of the 2015 10th Joint Meeting on Foundations of Software Engineering*, ser. ESEC/FSE 2015. New York, NY, USA: Association for Computing Machinery, 2015, p. 166–178. [Online]. Available: https://doi.org/10.1145/2786805.2786811

[10] S. Mechtaev, J. Yi, and A. Roychoudhury, "Angelix: Scalable multiline program patch synthesis via symbolic analysis," in *Proceedings of the 38th International Conference on Software Engineering*, ser. ICSE '16. New York, NY, USA: Association for Computing Machinery, 2016, p. 691–701. [Online]. Available: https://doi.org/10.1145/2884781.2884807

[11] R. Just, D. Jalali, and M. D. Ernst, "Defects4j: A database of existing faults to enable controlled testing studies for java programs," in *Proceedings of the 2014 International Symposium on Software Testing and Analysis*, ser. ISSTA 2014. New York, NY, USA: Association for Computing Machinery, 2014, p. 437–440. [Online]. Available: https://doi.org/10.1145/2610384.2628055

[12] R. K. Saha, Y. Lyu, W. Lam, H. Yoshida, and M. R. Prasad, "Bugs.jar: A large-scale, diverse dataset of real-world java bugs," in *Proceedings of the 15th International Conference on Mining Software Repositories*, ser. MSR '18. New York, NY, USA: Association for Computing Machinery, 2018, p. 10–13. [Online]. Available: https://doi.org/10.1145/3196398.3196473

[13] C. Le Goues, N. Holtschulte, E. K. Smith, Y. Brun, P. Devanbu, S. Forrest, and W. Weimer, "The manybugs and introclass benchmarks for automated repair of c programs," *IEEE Transactions on Software Engineering*, vol. 41, no. 12, pp. 1236–1256, 2015.

[14] S. Kirbas, E. Windels, O. McBello, K. Kells, M. Pagano, R. Szalanski, V. Nowack, E. R. Winter, S. Counsell, D. Bowes, T. Hall, S. Haraldsson, and J. Woodward, "On the introduction of automatic program repair in bloomberg," *IEEE Software*, vol. 38, no. 4, pp. 43–51, 2021.

[15] J. Bader, A. Scott, M. Pradel, and S. Chandra, "Getafix: Learning to fix bugs automatically," *Proc. ACM Program. Lang.*, vol. 3, no. OOPSLA, oct 2019. [Online]. Available: https://doi.org/10.1145/3360585

[16] A. Marginean, J. Bader, S. Chandra, M. Harman, Y. Jia, K. Mao, A. Mols, and A. Scott, "Sapfix: Automated end-to-end repair at scale," in *Proceedings of the 41st International Conference on Software Engineering: Software Engineering in Practice*, ser. ICSE-SEIP '19. IEEE Press, 2019, p. 269–278. [Online]. Available: https://doi.org/10.1109/ICSE-SEIP.2019.00039

[17] K. Liu, L. Li, A. Koyuncu, D. Kim, Z. Liu, J. Klein, and T. F. Bissyandé, "A critical review on the evaluation of automated program repair systems," *J. Syst. Softw.*, vol. 171, p. 110817, 2021. [Online]. Available: https://doi.org/10.1016/j.jss.2020.110817

[18] R. Schwartz, J. Dodge, N. A. Smith, and O. Etzioni, "Green ai," *Communications of the ACM*, vol. 63, no. 12, pp. 54–63, 2020.

[19] K. Liu, S. Wang, A. Koyuncu, K. Kim, T. F. Bissyandé, D. Kim, P. Wu, J. Klein, X. Mao, and Y. L. Traon, "On the efficiency of test suite based program repair: A systematic assessment of 16 automated repair systems for java programs," in *ICSE '20*. New York, NY, USA: ACM, 2020, pp. 615–627. [Online]. Available: https://doi.org/10.1145/3377811.3380338

[20] H. Ye, M. Martinez, and M. Monperrus, "Neural program repair with execution-based backpropagation," in *Proceedings of the 44th International Conference on Software Engineering*, ser. ICSE '22. New York, NY, USA: Association for Computing Machinery, 2022, p. 1506–1518. [Online]. Available: https://doi.org/10.1145/3510003.3510222

[21] M. Monperrus, S. Urli, T. Durieux, M. Martinez, B. Baudry, and L. Seinturier, "Repairnator patches programs automatically," *Ubiquity*, vol. July, no. 2, pp. 1–12, Jul. 2019. [Online]. Available: https://inria.hal.science/hal-02267512

[22] M. Beller, G. Gousios, and A. Zaidman, "Oops, my tests broke the build: An explorative analysis of travis ci with github," in *MSR*, 2017, pp. 356–367.

[23] R. Solingen, V. Basili, G. Caldiera, and D. Rombach, *Goal Question Metric (GQM) Approach*, 01 2002.

[24] A. Ghanbari, S. Benton, and L. Zhang, *Practical Program Repair via Bytecode Mutation*. New York, NY, USA: Association for Computing Machinery, 2019, p. 19–30. [Online]. Available: https://doi.org/10.1145/3293882.3330559

[25] Y. Qi, X. Mao, Y. Lei, and C. Wang, "Using automated program repair for evaluating the effectiveness of fault localization techniques," in *Proceedings of the 2013 International Symposium on Software Testing and Analysis*, ser. ISSTA 2013. New York, NY, USA: Association for Computing Machinery, 2013, p. 191–201. [Online]. Available: https://doi.org/10.1145/2483760.2483785

[26] M. Motwani, S. Sankaranarayanan, R. Just, and Y. Brun, "Do Automated Program Repair Techniques Repair Hard and Important Bugs?" *Empirical Software Engineering (EMSE)*, vol. 23, no. 5, pp. 2901–2947, October 2018, DOI: 10.1007/s10664-017-9550-0.

[27] K. Liu, A. Koyuncu, D. Kim, and T. F. Bissyandé, "TBar: Revisiting template-based automated program repair," in *Proceedings of the 28th ACM SIGSOFT International Symposium on Software Testing and Analysis*, ser. ISSTA. ACM, 2019, pp. 31–42.

[28] J. Jiang, Y. Xiong, H. Zhang, Q. Gao, and X. Chen, "Shaping program repair space with existing patches and similar code," in *Proceedings of the 27th ACM SIGSOFT International Symposium on Software Testing and Analysis*, ser. ISSTA 2018. New York, NY, USA: Association for Computing Machinery, 2018, p. 298–309.

[29] D. Fried, A. Aghajanyan, J. Lin, S. Wang, E. Wallace, F. Shi, R. Zhong, W. tau Yih, L. Zettlemoyer, and M. Lewis, "Incoder: A generative model for code infilling and synthesis," 2023.

[30] E. Nijkamp, B. Pang, H. Hayashi, L. Tu, H. Wang, Y. Zhou, S. Savarese, and C. Xiong, "Codegen: An open large language model for code with multi-turn program synthesis," 2023.

[31] N. Jiang, K. Liu, T. Lutellier, and L. Tan, "Impact of code language models on automated program repair," in *2023 IEEE/ACM 45th International Conference on Software Engineering (ICSE)*. Los Alamitos, CA, USA: IEEE Computer Society, may 2023, pp. 1430–1442. [Online]. Available: https://doi.ieeecomputersociety.org/10.1109/ICSE48619.2023.00125

Just Accepted

[32] A. Silva, S. Fang, and M. Monperrus, "Repairllama: Efficient representations and fine-tuned adapters for program repair," arXiv, Tech. Rep. 2312.15698. [Online]. Available: http://arxiv.org/abs/2312.15698

[33] B. Roziere, J. Gehring, F. Gloeckle, S. Sootla, I. Gat, X. E. Tan, Y. Adi, J. Liu, R. Sauvestre, T. Remez *et al.*, "Code llama: Open foundation models for code," *arXiv preprint arXiv:2308.12950*, 2023.

[34] E. J. Hu, Y. Shen, P. Wallis, Z. Allen-Zhu, Y. Li, S. Wang, L. Wang, and W. Chen, "Lora: Low-rank adaptation of large language models," 2021. [Online]. Available: https://arxiv.org/abs/2106.09685

[35] M. Martinez, S. Martínez-Fernández, and X. Franch, "Energy consumption of automated program repair," in *ICSE-Companion '24, 2024 IEEE/ACM 46th International Conference on Software Engineering: Companion Proceedings, April 14–20, 2024 Lisbon, Portugal*. IEEE/ACM, 2024. [Online]. Available: https://doi.org/10.1145/3639478.3643114

[36] M. Monperrus, "The living review on automated program repair," HAL/archives-ouvertes.fr, Tech. Rep. hal-01956501, 2018.

[37] K. Huang, Z. Xu, S. Yang, H. Sun, X. Li, Z. Yan, and Y. Zhang, "A survey on automated program repair techniques," 2023.

[38] H. Eladawy, C. Le Goues, and Y. Brun, "Automated program repair, what is it good for? not absolutely nothing!" in *Proceedings of the IEEE/ACM 46th International Conference on Software Engineering*, ser. ICSE '24. New York, NY, USA: Association for Computing Machinery, 2024. [Online]. Available: https://doi.org/10.1145/3597503.3639095

[39] Q. Zhang, C. Fang, Y. Ma, W. Sun, and Z. Chen, "A survey of learning-based automated program repair," *ACM Trans. Softw. Eng. Methodol.*, vol. 33, no. 2, dec 2023. [Online]. Available: https://doi.org/10.1145/3631974

[40] D. Kim, J. Nam, J. Song, and S. Kim, "Automatic patch generation learned from human-written patches," in *Proceedings of the 2013 International Conference on Software Engineering*, ser. ICSE '13. IEEE Press, 2013, p. 802–811.

[41] M. Wen, J. Chen, R. Wu, D. Hao, and S.-C. Cheung, "Context-aware patch generation for better automated program repair," in *Proceedings of the 40th International Conference on Software Engineering*, ser. ICSE '18. New York, NY, USA: Association for Computing Machinery, 2018, p. 1–11. [Online]. Available: https://doi.org/10.1145/3180155.3180233

[42] X. B. D. Le, D. Lo, and C. Le Goues, "History driven program repair," in *2016 IEEE 23rd International Conference on Software Analysis, Evolution, and Reengineering (SANER)*, vol. 1, 2016, pp. 213–224.

[43] A. Koyuncu, K. Liu, T. F. Bissyandé, D. Kim, J. Klein, M. Monperrus, and Y. Le Traon, "FixMiner: Mining relevant fix patterns for automated program repair," *Empirical Software Engineering*, pp. 1–45, 2020.

[44] M. Martinez and M. Monperrus, "Mining software repair models for reasoning on the search space of automated program fixing," *Empirical Softw. Engg.*, vol. 20, no. 1, p. 176–205, feb 2015. [Online]. Available: https://doi.org/10.1007/s10664-013-9282-8

[45] F. Long and M. Rinard, "Automatic patch generation by learning correct code," *SIGPLAN Not.*, vol. 51, no. 1, p. 298–312, jan 2016. [Online]. Available: https://doi.org/10.1145/2914770.2837617

[46] H. D. T. Nguyen, D. Qi, A. Roychoudhury, and S. Chandra, "Semfix: Program repair via semantic analysis," in *Proceedings of the 2013 International Conference on Software Engineering*, ser. ICSE '13. IEEE Press, 2013, p. 772–781.

[47] J. Xuan, M. Martinez, F. DeMarco, M. Clément, S. L. Marcote, T. Durieux, D. L. Berre, and M., "Nopol: Automatic repair of conditional statement bugs in Java programs," *IEEE Transactions on Software Engineering*, vol. 43, no. 1, pp. 34–55, 2017.

[48] Z. Chen, S. Kommrusch, M. Tufano, L.-N. Pouchet, D. Poshyvanyk, and M. Monperrus, "Sequencer: Sequence-to-sequence learning for end-to-end program repair," *IEEE Transactions on Software Engineering*, vol. 47, no. 9, pp. 1943–1959, 2021.

[49] T. Lutellier, H. V. Pham, L. Pang, Y. Li, M. Wei, and L. Tan, "Coconut: Combining context-aware neural translation models using ensemble for program repair," in *Proceedings of the 29th ACM SIGSOFT International Symposium on Software Testing and Analysis*, ser. ISSTA 2020. New York, NY, USA: Association for Computing Machinery, 2020, p. 101–114. [Online]. Available: https://doi.org/10.1145/3395363.3397369

[50] Q. Zhu, Z. Sun, Y.-a. Xiao, W. Zhang, K. Yuan, Y. Xiong, and L. Zhang, "A syntax-guided edit decoder for neural program repair," in *Proceedings of the 29th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering*, ser. ESEC/FSE 2021. New York, NY, USA: Association for Computing Machinery, 2021, p. 341–353. [Online]. Available: https://doi.org/10.1145/3468264.3468544

[51] H. Ye, M. Martinez, X. Luo, T. Zhang, and M. Monperrus, "Selfapr: Self-supervised program repair with test execution diagnostics," in *Proceedings of the 37th IEEE/ACM International Conference on Automated Software Engineering*, ser. ASE '22. New York, NY, USA: Association for Computing Machinery, 2023. [Online]. Available: https://doi.org/10.1145/3551349.3556926

[52] N. Jiang, T. Lutellier, Y. Lou, L. Tan, D. Goldwasser, and X. Zhang, "KNOD: domain knowledge distilled tree decoder for automated program repair," in *45th IEEE/ACM International Conference on Software Engineering, ICSE 2023, Melbourne, Australia, May 14-20, 2023*. IEEE, 2023, pp. 1251–1263. [Online]. Available: https://doi.org/10.1109/ICSE48619.2023.00111

[53] Z. Fan, X. Gao, M. Mirchev, A. Roychoudhury, and S. Tan, "Automated repair of programs from large language models," in *2023 IEEE/ACM 45th International Conference on Software Engineering (ICSE)*. Los Alamitos, CA, USA: IEEE Computer Society, may 2023, pp. 1469–1481. [Online]. Available: https://doi.ieeecomputersociety.org/10.1109/ICSE48619.2023.00128

[54] D. Sobania, M. Briesch, C. Hanna, and J. Petke, "An analysis of the automatic bug fixing performance of chatgpt," in *2023 IEEE/ACM International Workshop on Automated Program Repair (APR)*. Los Alamitos, CA, USA: IEEE Computer Society, may 2023, pp. 23–30. [Online]. Available: https://doi.ieeecomputersociety.org/10.1109/APR59189.2023.00012

Just Accepted

[55] D. Williams, J. Callan, S. Kirbas, S. Mechtaev, J. Petke, T. Prideaux-Ghee, and F. Sarro, "User-centric deployment of automated program repair at bloomberg," in *Proceedings of the 46th International Conference on Software Engineering: Software Engineering in Practice*, ser. ICSE-SEIP '24. New York, NY, USA: Association for Computing Machinery, 2024, p. 81–91. [Online]. Available: https://doi.org/10.1145/3639477.3639756

[56] F. N. Meem, J. Smith, and B. Johnson, "Exploring experiences with automated program repair in practice," in *2024 IEEE/ACM 46th International Conference on Software Engineering (ICSE)*, 2024, pp. 1047–1057.

[57] M. Martinez and M. Monperrus, "ASTOR: A program repair library for Java (demo)," in *Proceedings of the 25th International Symposium on Software Testing and Analysis (ISSTA)*. ACM, 2016, pp. 441–444.

[58] C. Le Goues, M. Dewey-Vogt, S. Forrest, and W. Weimer, "A systematic study of automated program repair: Fixing 55 out of 105 bugs for $8 each," in *2012 34th International Conference on Software Engineering (ICSE)*, 2012, pp. 3–13.

[59] E. K. Smith, E. T. Barr, C. Le Goues, and Y. Brun, "Is the cure worse than the disease? overfitting in automated program repair," in *Proceedings of the 2015 10th Joint Meeting on Foundations of Software Engineering*, 2015, pp. 532–543.

[60] M. Martinez, T. Durieux, R. Sommerard, J. Xuan, and M. Monperrus, "Automatic repair of real bugs in java: A large-scale experiment on the defects4j dataset," *Empirical Software Engineering*, vol. 22, no. 4, pp. 1936–1964, 2017.

[61] Z. Qi, F. Long, S. Achour, and M. Rinard, "An analysis of patch plausibility and correctness for generate-and-validate patch generation systems," in *Proceedings of the 2015 International Symposium on Software Testing and Analysis*, 2015, pp. 24–36.

[62] X. B. D. Le, F. Thung, D. Lo, and C. Le Goues, "Overfitting in semantics-based automated program repair," *Empirical Software Engineering*, vol. 23, no. 5, pp. 3007–3033, 2018.

[63] H. Ye, M. Martinez, and M. Monperrus, "Automated patch assessment for program repair at scale," *Empir. Softw. Eng.*, vol. 26, no. 2, p. 20, 2021. [Online]. Available: https://doi.org/10.1007/s10664-020-09920-w

[64] Y. Xiong, X. Liu, M. Zeng, L. Zhang, and G. Huang, "Identifying patch correctness in test-based program repair," in *Proceedings of the 40th International Conference on Software Engineering, ICSE 2018, Gothenburg, Sweden, May 27 - June 03, 2018*, M. Chaudron, I. Crnkovic, M. Chechik, and M. Harman, Eds. ACM, 2018, pp. 789–799. [Online]. Available: https://doi.org/10.1145/3180155.3180182

[65] Q. Xin and S. P. Reiss, "Identifying test-suite-overfitted patches through test case generation," in *Proceedings of the 26th ACM SIGSOFT International Symposium on Software Testing and Analysis*, ser. ISSTA 2017. New York, NY, USA: Association for Computing Machinery, 2017, p. 226–236. [Online]. Available: https://doi.org/10.1145/3092703.3092718

[66] M. Martinez, M. Kechagia, A. Perera, J. Petke, F. Sarro, and A. Aleti, "Test-based patch clustering for automatically-generated patches assessment," *Empirical Software Engineering*, vol. 29, no. 5, p. 116, 2024. [Online]. Available: https://doi.org/10.1007/s10664-024-10503-2

[67] S. Wang, M. Wen, B. Lin, H. Wu, Y. Qin, D. Zou, X. Mao, and H. Jin, "Automated patch correctness assessment: How far are we?" in *Proceedings of the 35th IEEE/ACM International Conference on Automated Software Engineering*, ser. ASE '20. New York, NY, USA: Association for Computing Machinery, 2020, pp. 968––980.

[68] J. Petke, M. Martinez, M. Kechagia, A. Aleti, and F. Sarro, "The patch overfitting problem in automated program repair: Practical magnitude and a baseline for realistic benchmarking," in *Companion Proceedings of the 32nd ACM International Conference on the Foundations of Software Engineering*, ser. FSE 2024. New York, NY, USA: Association for Computing Machinery, 2024, p. 452–456. [Online]. Available: https://doi.org/10.1145/3663529.3663776

[69] H. Ye, J. Gu, M. Martinez, T. Durieux, and M. Monperrus, "Automated classification of overfitting patches with statically extracted code features," *IEEE Transactions on Software Engineering*, 2021.

[70] A. Ghanbari and A. Marcus, "Patch correctness assessment in automated program repair based on the impact of patches on production and test code," in *Proceedings of the 31st ACM SIGSOFT International Symposium on Software Testing and Analysis*, ser. ISSTA 2022. New York, NY, USA: Association for Computing Machinery, 2022, p. 654–665. [Online]. Available: https://doi.org/10.1145/3533767.3534368

[71] B. Lin, S. Wang, M. Wen, and X. Mao, "Context-aware code change embedding for better patch correctness assessment," *ACM Trans. Softw. Eng. Methodol.*, vol. 31, no. 3, May 2022. [Online]. Available: https://doi.org/10.1145/3505247

[72] H. Tian, K. Liu, A. K. Kaboré, A. Koyuncu, L. Li, J. Klein, and T. F. Bissyandé, "Evaluating representation learning of code changes for predicting patch correctness in program repair," in *Proceedings of the 35th IEEE/ACM International Conference on Automated Software Engineering*, ser. ASE '20. New York, NY, USA: Association for Computing Machinery, 2021, p. 981–992. [Online]. Available: https://doi.org/10.1145/3324884.3416532

[73] A. Ghanbari, "Objsim: Lightweight automatic patch prioritization via object similarity," in *Proceedings of the 29th ACM SIGSOFT International Symposium on Software Testing and Analysis*, ser. ISSTA 2020. New York, NY, USA: Association for Computing Machinery, 2020, p. 541–544. [Online]. Available: https://doi.org/10.1145/3395363.3404362

[74] F. Palomba, D. Di Nucci, A. Panichella, A. Zaidman, and A. De Lucia, "On the impact of code smells on the energy consumption of mobile applications," *Information and Software Technology*, vol. 105, pp. 43–55, 2019. [Online]. Available: https://www.sciencedirect.com/science/article/pii/S0950584918301678

[75] E. Guégain, C. Quinton, and R. Rouvoy, "On reducing the energy consumption of software product lines," in *Proceedings of the 25th ACM International Systems and Software Product Line Conference - Volume A*, ser. SPLC '21. New York, NY, USA: Association for Computing Machinery, 2021, p. 89–99. [Online]. Available: https://doi-org.recursos.biblioteca.upc.edu/10.1145/3461001.3471142

Just Accepted

[76] R. C. Castanyer, S. Martínez-Fernández, and X. Franch, "Which design decisions in ai-enabled mobile applications contribute to greener ai?" *Empirical Software Engineering*, vol. 29, no. 1, p. 2, 2023. [Online]. Available: https://doi.org/10.1007/s10664-023-10407-7

[77] E. Strubell, A. Ganesh, and A. McCallum, "Energy and policy considerations for deep learning in nlp," *arXiv preprint arXiv:1906.02243*, 2019.

[78] S. Georgiou, M. Kechagia, T. Sharma, F. Sarro, and Y. Zou, "Green ai: Do deep learning frameworks have different costs?" ACM: Association for Computing Machinery, 2022.

[79] M. Hort, A. Grishina, and L. Moonen, "An exploratory literature study on sharing and energy use of language models for source code," in *2023 ACM/IEEE International Symposium on Empirical Software Engineering and Measurement (ESEM)*. Los Alamitos, CA, USA: IEEE Computer Society, oct 2023, pp. 1–12. [Online]. Available: https://doi.ieeecomputersociety.org/10.1109/ESEM56168.2023.10304803

[80] N. Jiang, T. Lutellier, and L. Tan, "Cure: Code-aware neural machine translation for automatic program repair," in *2021 IEEE/ACM 43rd International Conference on Software Engineering (ICSE)*, 2021, pp. 1161–1173.

[81] B. Berabi, J. He, V. Raychev, and M. Vechev, "Tfix: Learning to fix coding errors with a text-to-text transformer," in *Proceedings of the 38th International Conference on Machine Learning*, ser. Proceedings of Machine Learning Research, M. Meila and T. Zhang, Eds., vol. 139. PMLR, 18–24 Jul 2021, pp. 780–791. [Online]. Available: https://proceedings.mlr.press/v139/berabi21a.html

[82] R. Gupta, A. Kanade, and S. Shevade, "Deep reinforcement learning for syntactic error repair in student programs," *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 33, no. 01, pp. 930–937, Jul. 2019. [Online]. Available: https://ojs.aaai.org/index.php/AAAI/article/view/3882

[83] J. Shi, Z. Yang, and D. Lo, "Efficient and green large language models for software engineering: Vision and the road ahead," 2024. [Online]. Available: https://arxiv.org/abs/2404.04566

[84] I. Manotas, C. Bird, R. Zhang, D. Shepherd, C. Jaspan, C. Sadowski, L. Pollock, and J. Clause, "An empirical study of practitioners' perspectives on green software engineering," in *Proceedings of the 38th International Conference on Software Engineering*, ser. ICSE '16. New York, NY, USA: Association for Computing Machinery, 2016, p. 237–248. [Online]. Available: https://doi.org/10.1145/2884781.2884810

[85] C. Pang, A. Hindle, B. Adams, and A. E. Hassan, "What do programmers know about software energy consumption?" *IEEE Software*, vol. 33, no. 3, pp. 83–89, 2016.

[86] C. Wohlin, P. Runeson, M. Höst, M. C. Ohlsson, B. Regnell, A. Wesslén *et al.*, *Experimentation in software engineering*. Springer, vol. 236.

[87] T. Durieux, F. Madeiral, M. Martinez, and R. Abreu, "Empirical review of java program repair tools: A large-scale experiment on 2,141 bugs and 23,551 repair attempts," in *Proceedings of the 2019 27th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering*. ACM, 2019, pp. 302–313.

[88] K. Liu, A. Koyuncu, D. Kim, and T. F. Bissyandé, "AVATAR: Fixing semantic bugs with fix patterns of static analysis violations," *IEEE 26th International Conference on Software Analysis, Evolution and Reengineering (SANER)*, pp. 1–12, 2018.

[89] T. Durieux and M. Monperrus, "Dynamoth: Dynamic code synthesis for automatic program repair," in *2016 IEEE/ACM 11th International Workshop in Automation of Software Test (AST)*, 2016, pp. 85–91.

[90] K. Liu, A. Koyuncu, T. F. Bissyandé, D. Kim, J. Klein, and Y. Le Traon, "You cannot fix what you cannot find! an investigation of fault localization bias in benchmarking automated program repair systems," in *2019 12th IEEE Conference on Software Testing, Validation and Verification (ICST)*, 2019, pp. 102–113.

[91] Y. Xiong, J. Wang, R. Yan, J. Zhang, S. Han, G. Huang, and L. Zhang, "Precise condition synthesis for program repair," in *Proceedings of the 39th International Conference on Software Engineering*, ser. ICSE '17. IEEE Press, 2017, p. 416–426.

[92] M. Martinez and M. Monperrus, "Ultra-large repair search space with automatically mined templates: The cardumen mode of astor," in *International Symposium on Search Based Software Engineering*. Springer, 2018, pp. 65–86.

[93] Y. Li, S. Wang, and T. N. Nguyen, "Dear: A novel deep learning-based approach for automated program repair," in *Proceedings of the 44th International Conference on Software Engineering*, ser. ICSE '22. New York, NY, USA: Association for Computing Machinery, 2022, p. 511–523. [Online]. Available: https://doi.org/10.1145/3510003.3510177

[94] N. Jiang, T. Lutellier, and L. Tan, "Cure: Code-aware neural machine translation for automatic program repair," in *2021 IEEE/ACM 43rd International Conference on Software Engineering (ICSE)*, 2021, pp. 1161–1173.

[95] Q. Zhang, T. Zhang, J. Zhai, C. Fang, B. Yu, W. Sun, and Z. Chen, "A critical review of large language model on software engineering: An example from chatgpt and automated program repair," 2023.

[96] Y. Wang, W. Wang, S. Joty, and S. C. Hoi, "CodeT5: Identifier-aware unified pre-trained encoder-decoder models for code understanding and generation," in *Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing*. Online and Punta Cana, Dominican Republic: Association for Computational Linguistics, Nov. 2021, pp. 8696–8708. [Online]. Available: https://aclanthology.org/2021.emnlp-main.685

[97] W. Ahmad, S. Chakraborty, B. Ray, and K.-W. Chang, "Unified pre-training for program understanding and generation," in *Proceedings of the 2021 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*. Online: Association for Computational Linguistics, Jun. 2021, pp. 2655–2668. [Online]. Available: https://www.aclweb.org/anthology/2021.naacl-main.211

Just Accepted

[98] L. Cruz, "Green software engineering done right: a scientific guide to set up energy efficiency experiments," http://luiscruz.github.io/2021/10/10/scientific-guide.html, 2021, blog post.

[99] A. Ghanbari and L. Zhang, "Prapr: Practical program repair via bytecode mutation," in *2019 34th IEEE/ACM International Conference on Automated Software Engineering (ASE)*, 2019, pp. 1118–1121.

[100] S. Elbaum, A. Malishevsky, and G. Rothermel, "Test case prioritization: a family of empirical studies," *IEEE Transactions on Software Engineering*, vol. 28, no. 2, pp. 159–182, 2002.

[101] B. Baudry, Z. Chen, K. Etemadi, H. Fu, D. Ginelli, S. Kommrusch, M. Martinez, M. Monperrus, J. Ron, H. Ye, and Z. Yu, "A software-repair robot based on continual learning," *IEEE Software*, vol. 38, no. 4, pp. 28–35, 2021.

[102] D. Ramos, C. Mamede, K. Jain, P. Canelas, C. Gamboa, and C. L. Goues, "Are large language models memorizing bug benchmarks?" 2024. [Online]. Available: https://arxiv.org/abs/2411.13323

[103] Z. Yang, Z. Zhao, C. Wang, J. Shi, D. Kim, D. Han, and D. Lo, "Unveiling memorization in code models," in *Proceedings of the IEEE/ACM 46th International Conference on Software Engineering*, ser. ICSE '24. New York, NY, USA: Association for Computing Machinery, 2024. [Online]. Available: https://doi.org/10.1145/3597503.3639074

[104] V. Bolón-Canedo, L. Morán-Fernández, B. Cancela, and A. Alonso-Betanzos, "A review of green artificial intelligence: Towards a more sustainable future," *Neurocomputing*, vol. 599, p. 128096, 2024. [Online]. Available: https://www.sciencedirect.com/science/article/pii/S0925231224008671

[105] J. Xuan, B. Cornu, M. Martinez, B. Baudry, L. Seinturier, and M. Monperrus, "B-refactoring: Automatic test code refactoring to improve dynamic analysis," *Information and Software Technology*, vol. 76, pp. 65–80, 2016. [Online]. Available: https://www.sciencedirect.com/science/article/pii/S0950584916300714

[106] F. Long and M. Rinard, "An analysis of the search spaces for generate and validate patch generation systems," in *Proceedings of the 38th International Conference on Software Engineering*, ser. ICSE '16. New York, NY, USA: Association for Computing Machinery, 2016, p. 702–713. [Online]. Available: https://doi.org/10.1145/2884781.2884872

[107] Y. Li, Y. Yu, C. Liang, P. He, N. Karampatziakis, W. Chen, and T. Zhao, "Loftq: Lora-fine-tuning-aware quantization for large language models," *arXiv preprint arXiv:2310.08659*, 2023.

[108] X. Wei, S. K. Gonugondla, S. Wang, W. Ahmad, B. Ray, H. Qian, X. Li, V. Kumar, Z. Wang, Y. Tian, Q. Sun, B. Athiwaratkun, M. Shang, M. K. Ramanathan, P. Bhatia, and B. Xiang, "Towards greener yet powerful code generation via quantization: An empirical study," in *Proceedings of the 31st ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering*, ser. ESEC/FSE 2023. New York, NY, USA: Association for Computing Machinery, 2023, p. 224–236. [Online]. Available: https://doi.org/10.1145/3611643.3616302

[109] J. Shi, Z. Yang, B. Xu, H. J. Kang, and D. Lo, "Compressing pre-trained models of code into 3 mb," in *Proceedings of the 37th IEEE/ACM International Conference on Automated Software Engineering*, ser. ASE '22. New York, NY, USA: Association for Computing Machinery, 2023. [Online]. Available: https://doi.org/10.1145/3551349.3556964

[110] J. Shi, Z. Yang, H. J. Kang, B. Xu, J. He, and D. Lo, "Greening large language models of code," in *Proceedings of the 46th International Conference on Software Engineering: Software Engineering in Society*, ser. ICSE-SEIS'24. New York, NY, USA: Association for Computing Machinery, 2024, p. 142–153. [Online]. Available: https://doi.org/10.1145/3639475.3640097

[111] A. Silva and M. Monperrus, "Repairbench: Leaderboard of frontier models for program repair," 2024. [Online]. Available: https://arxiv.org/abs/2409.18952

[112] F. Durán, M. Martinez, P. Lago, and S. Martínez-Fernández, "Energy consumption of code small language models serving with runtime engines and execution providers," 2024. [Online]. Available: https://arxiv.org/abs/2412.15441

[113] M. Martinez, "The impact of hyperparameters on large language model inference performance: An evaluation of vllm and huggingface pipelines," 2024. [Online]. Available: https://arxiv.org/abs/2408.01050

[114] D. Hidvégi, K. Etemadi, S. Bobadilla, and M. Monperrus, "Cigar: Cost-efficient program repair with llms," 2024. [Online]. Available: https://arxiv.org/abs/2402.06598

[115] R. Rubei, A. Moussaid, C. di Sipio, and D. di Ruscio, "Prompt engineering and its implications on the energy consumption of large language models," 2025. [Online]. Available: https://arxiv.org/abs/2501.05899

[116] L. Chen, Y. Pei, and C. A. Furia, "Contract-based program repair without the contracts," in *Proceedings of the 32nd IEEE/ACM International Conference on Automated Software Engineering*, ser. ASE 2017. IEEE Press, 2017, p. 637–647.

[117] M. Smytzek, M. Eberlein, W. Kai, G. Lars, and A. Zeller, "FixKit: A Program Repair Collection for Python," 10 2024. [Online]. Available: https://publications.cispa.de/articles/conference_contribution/FixKit_A_Program_Repair_Collection_for_Python/27210285

[118] H. Ye and M. Monperrus, "Iter: Iterative neural repair for multi-location patches," in *Proceedings of the IEEE/ACM 46th International Conference on Software Engineering*, ser. ICSE '24. New York, NY, USA: Association for Computing Machinery, 2024. [Online]. Available: https://doi.org/10.1145/3597503.3623337

[119] C. S. Xia and L. Zhang, "Automated program repair via conversation: Fixing 162 out of 337 bugs for $0.42 each using chatgpt," in *Proceedings of the 33rd ACM SIGSOFT International Symposium on Software Testing and Analysis*, ser. ISSTA 2024. New York, NY, USA: Association for Computing Machinery, 2024, p. 819–831. [Online]. Available: https://doi.org/10.1145/3650212.3680323

Just Accepted